VELAGAPUDI RAMAKRISHNA SIDDHARTHA ENGINEERING COLLEGE

Department of Computer Science and Engineering

SALESFORCE CRM PROJECT DOCUMENTATION

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Submitted as part of the

Salesforce Certified Platform Developer Certification Program

Submitted By	KALUVA PRANEETH KUMAR
Register Number	238W1A05M1
Year / Semester	IV Year (Final Year)
Section	D
Programme	Salesforce Certified Platform Developer
Certification Partner	Smart Bridge
Salesforce Org	238w1a05m1.47fc9c8d16a7@agentforce.com
Date	September 2026

Table of Contents

Table of Contents 2
1. Project Overview 4
2. Objectives 4
2.1 Centralize Automotive Information 4
2.2 Improve Order Processing 4
2.3 Reduce Manual Effort 4
2.4 Maintain Data Accuracy 4
2.5 Simplify User Interaction 4
3. Phase 1 — Requirement Analysis and Planning 5
3.1 Business Requirements 5
3.2 Project Scope 5
3.3 Data Model 6
3.4 Security Model 7
4. Phase 2 — Salesforce Development: Backend & Configuration 7
4.1 Setup Environment & DevOps Workflow 7
4.2 Custom Objects 7
4.2.1 Custom Tabs 11
4.3 Fields 12
4.3.1 Representative Field-Creation Walkthrough 14
4.4 Object Relationships 15
4.5 Validation Rule — Vehicle Stock 16
4.6 Flow Automation 16
4.6.1 Auto Assign Dealer 16
4.6.2 Test Drive Reminder 18
4.7 Apex Classes and Triggers 19
4.7.1 Apex Classes 19
4.7.2 Apex Trigger 20
5. Phase 3 — UI/UX Development & Customization 21
5.1 Lightning App Setup via App Manager 21
5.2 Page Layouts & Dynamic Forms 23
5.3 User Management 23
5.4 Reports and Dashboards 24
5.5 Lightning Record Pages 24

6. Phase 4 — Data Migration, Testing & Security 25
6.1 Data Loading 25
6.2 Profiles 25
6.3 Roles and Role Hierarchy 26
6.4 Permission Sets 26
6.5
Test Case 1 — Vehicle Creation 27
Test Case 2 — Stock Validation on Order Confirmation 28
Test Case 3 — Test Drive Reminder Email 28
Test Case 4 — Test Drive Completion 29
7. Phase 5 — Deployment, Documentation & Maintenance 30
7.1 Deployment Strategy 30
7.2 Maintenance & Monitoring 30
7.3 Troubleshooting Approach 31
8. Conclusion 31

1. Project Overview

WhatNext Vision Motors is a purpose-built Salesforce CRM solution designed to support the core business operations of an automotive dealership network. The application brings together every stage of the vehicle lifecycle — from inventory and dealer management to customer engagement, ordering, test drives, and after-sales service — into a single, unified Salesforce workspace.

The platform provides a common workspace for managing information associated with vehicles, customers, dealers, vehicle orders, test drives, and service requests. Vehicle availability is validated before any order is confirmed, and the system automatically associates each order with the most appropriate dealer based on configured business logic.

Automation plays a central role in the solution: scheduled reminders are sent ahead of confirmed test drives, and pending orders are processed once stock becomes available. The Lightning Experience interface gives users organized navigation and purpose-built record pages, while reports and dashboards support ongoing business analysis.

2. Objectives

The project was developed with the following objectives:

2.1 Centralize Automotive Information

Establish a common Salesforce environment for maintaining vehicle, customer, dealer, order, test-drive, and service information.

2.2 Improve Order Processing

Provide a structured, validated method for creating and managing vehicle orders that accounts for real-time vehicle availability.

2.3 Reduce Manual Effort

Automate repetitive operations using Salesforce Flow and Apex, minimizing manual intervention in routine business processes.

2.4 Maintain Data Accuracy

Apply validation rules and programmatic checks to reduce incorrect or incomplete transactions.

2.5 Simplify User Interaction

Deliver a Lightning application with organized navigation, purpose-built page layouts, and relevant record information at every step.

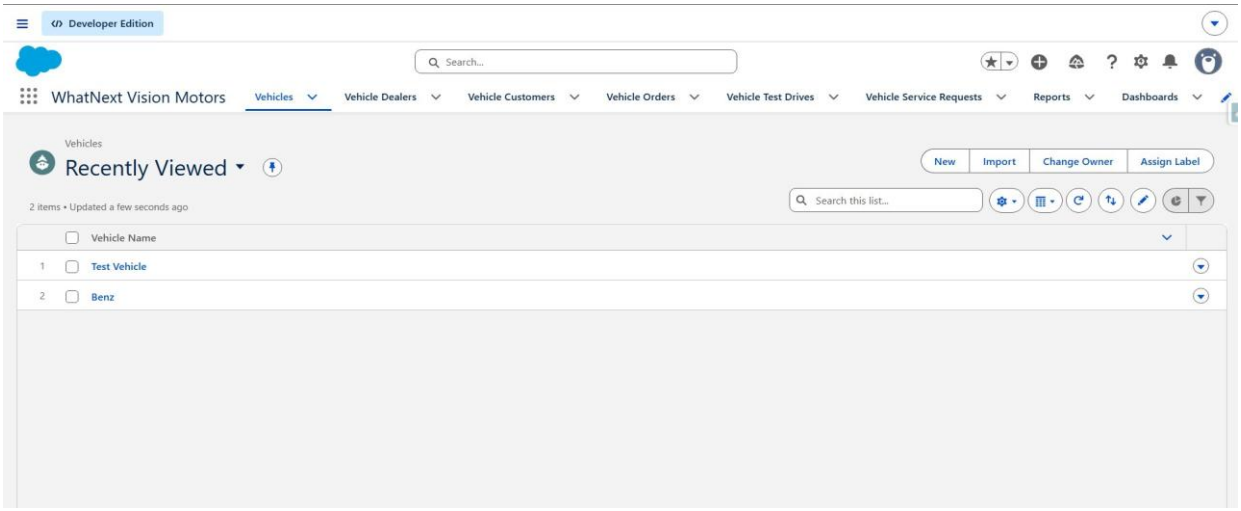


Figure 2.1 — The WhatNext Vision Motors Lightning App, showing the Vehicles tab and sample vehicle records.

3. Phase 1 — Requirement Analysis and Planning

The first phase focused on translating the business scenario into concrete Salesforce requirements. Key activities in this phase included requirement analysis, defining project scope, designing the data model, and planning the security model.

3.1 Business Requirements

Requirements were identified by studying the day-to-day activities performed by an automotive dealership organization. The system needed to maintain information about vehicles and customers while supporting activities performed both before and after a vehicle purchase. The major business requirements identified were:

- Store vehicle information (model, price, stock, availability).
- Maintain customer records and preferences.
- Maintain dealer information.
- Record and track vehicle orders.
- Associate orders with customers, vehicles, and dealers.
- Record and manage service requests raised by customers.

3.2 Project Scope

- **Vehicle Management:** Maintains essential vehicle information, including model details, pricing, stock availability, and related vehicle data.
- **Dealer Management:** Stores and manages dealership information to support vehicle sales and dealer-related activities.

- **Customer Management:** Maintains customer details and supports sales, test-drive, and service interactions.
- **Order and Test Drive Management:** Records vehicle purchase orders and manages scheduled test-drive activities by connecting customers with their selected vehicles.
- **Service Request Management:** Records customer service requirements and links them to the appropriate customer and vehicle records.

3.3 Data Model

The data model represents the major entities involved in the automotive CRM process. A vehicle can be associated with order and test-drive records; customer information can be connected to orders, test drives, and service requests; and dealer information can be associated with vehicle transactions. These relationships eliminate duplicate data entry and make related records easy to access from any linked record.

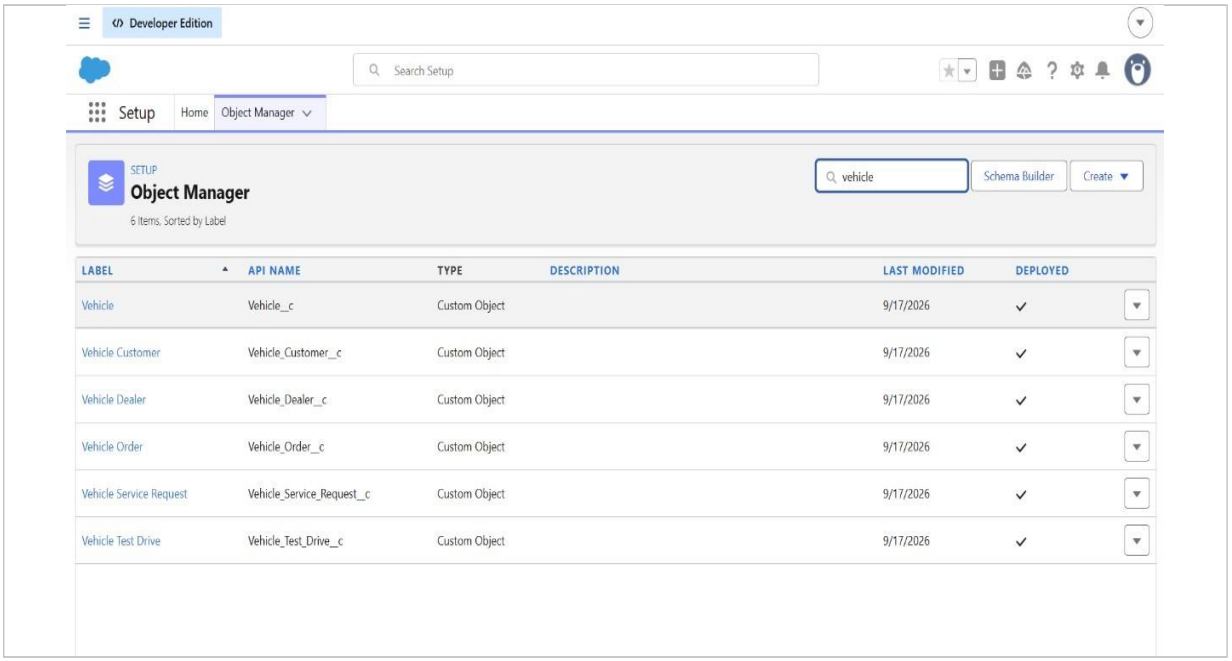


Figure 3.1 — Object Manager showing all six custom objects, deployed and ready for us

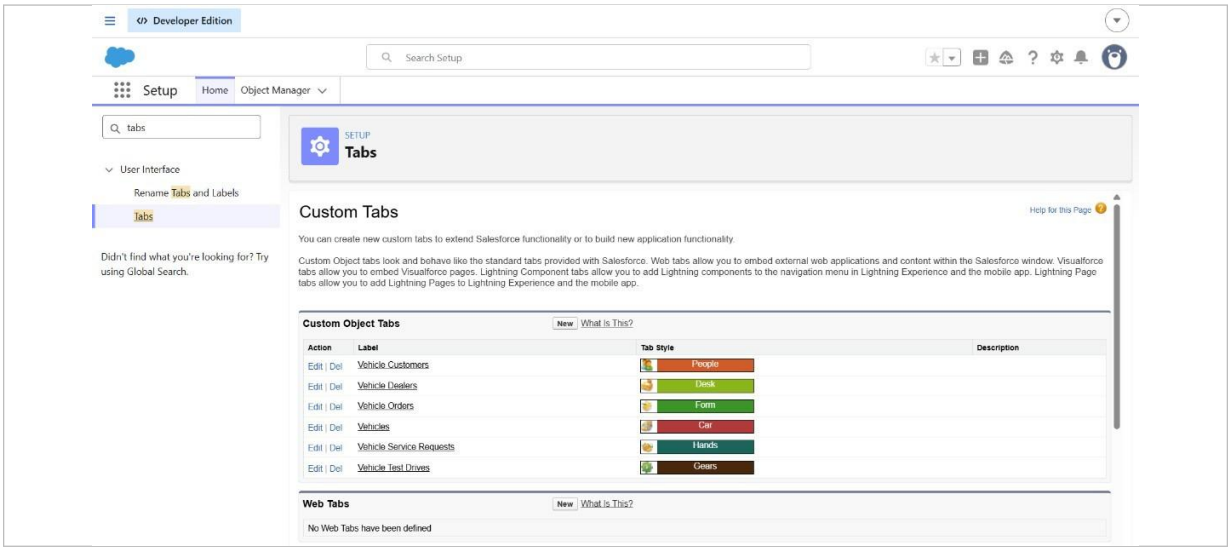


Figure 3.2 — Custom Tabs configured for each object, giving each entity its own navigable tab in the app.

3.4 Security Model

Security was considered as an integral part of the application design from the outset. The Salesforce security model applied to this project uses multiple, layered levels of access control:

- Profiles
- Roles and role hierarchy
- Permission sets
- Object- and field-level permissions
- Sharing rules

4. Phase 2 — Salesforce Development: Backend & Configuration

The second phase covered implementation of the required Salesforce data structures and backend functionality: custom objects, fields, validation rules, automation, and Apex.

4.1 Setup Environment & DevOps Workflow

The application was built on Salesforce Lightning Experience using a Developer Edition org. Salesforce provided the environment for creating custom objects, configuring fields, defining relationships, implementing declarative and programmatic automation, building reports, and managing security — all within a single connected workspace.

4.2 Custom Objects

The project uses six custom objects to represent the major business entities of the WhatNext Vision Motors automotive CRM system:

- **Vehicle:** Stores vehicle details such as model, price, stock quantity, availability, and dealer information.
- **Vehicle Dealer:** Maintains dealership details, including dealer name, location, city, and contact information.
- **Vehicle Customer:** Stores customer information required for vehicle purchases, test drives, and service activities.
- **Vehicle Order:** Records customer vehicle purchases and tracks the selected vehicle, customer, dealer, and order status.
- **Vehicle Test Drive:** Manages test-drive bookings, maintaining customer, vehicle, appointment date, and status details.
- **Vehicle Service Request:** Records customer service requirements and links the related vehicle, customer, and service status.

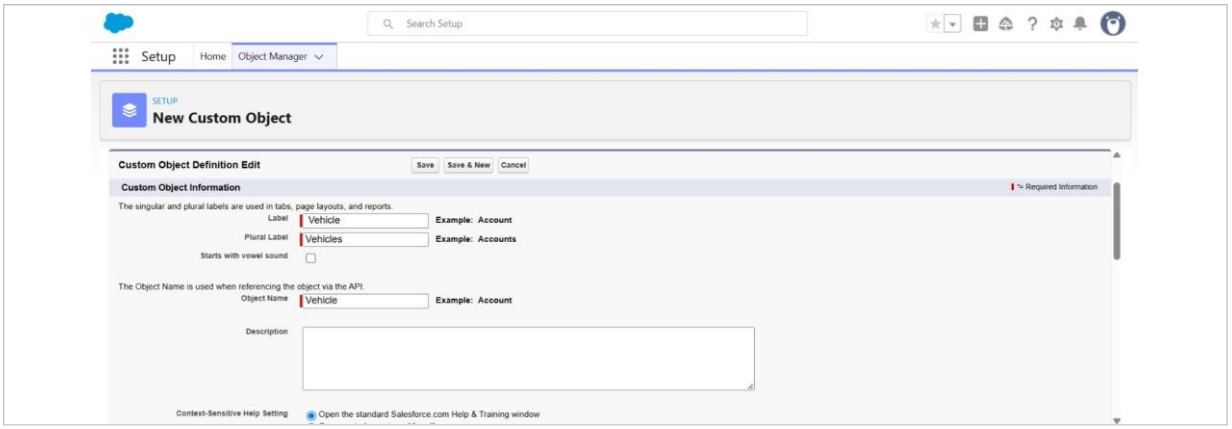


Figure 4.1 — Creating the Vehicle custom object: label, plural label, and API name.

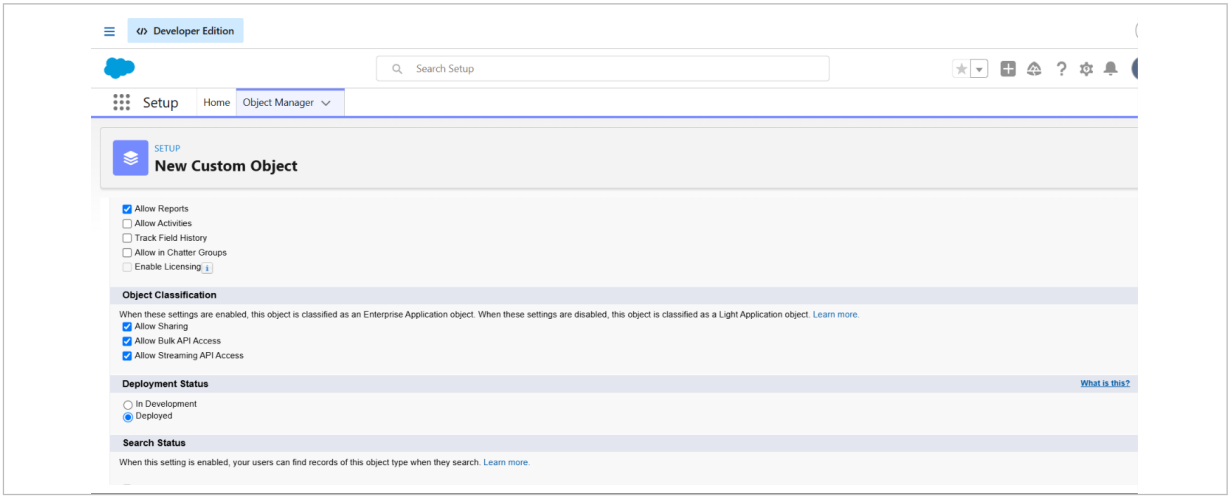


Figure 4.2 — Object classification and deployment settings for the Vehicle object.

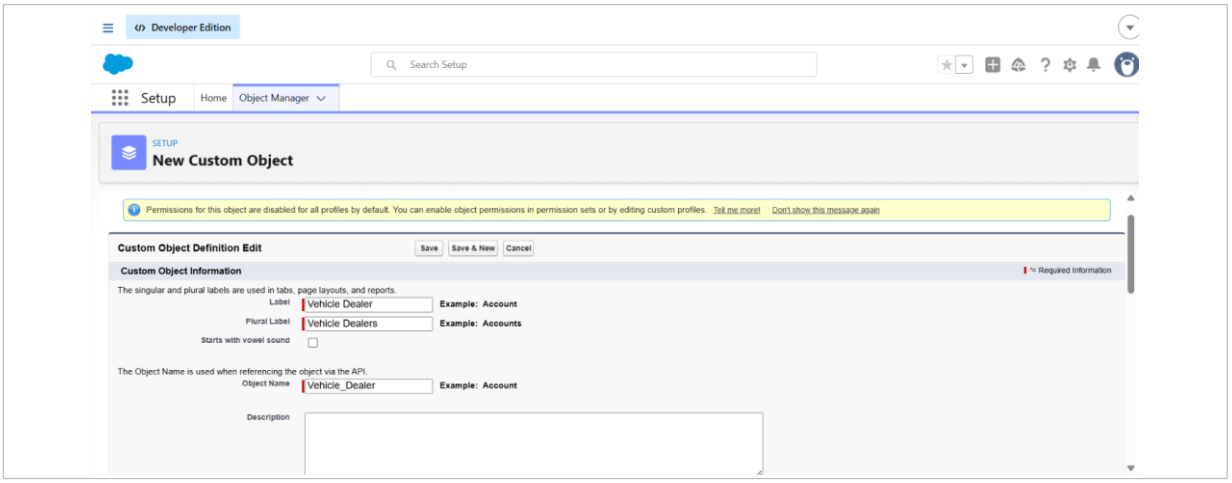


Figure 4.3 — Creating the Vehicle Dealer custom object.

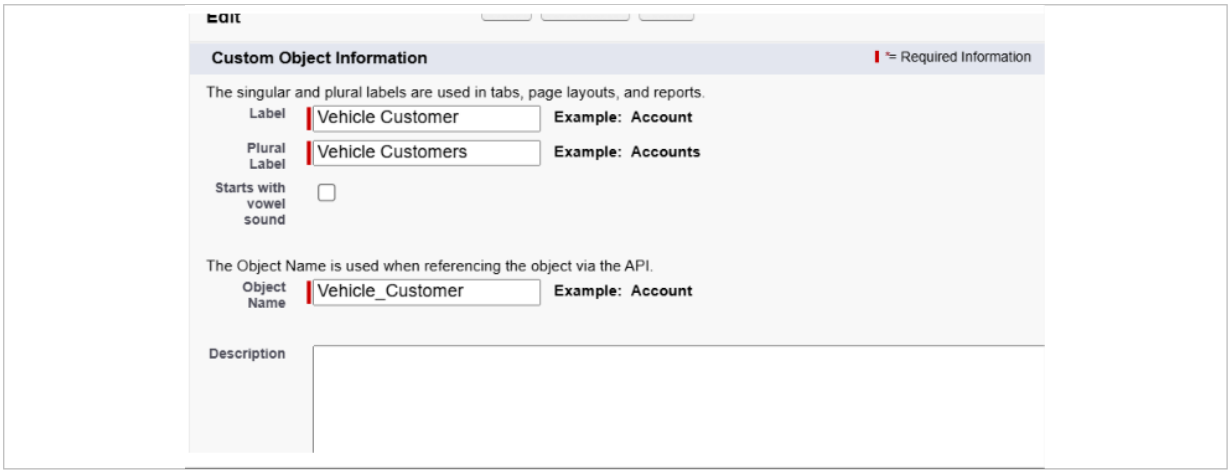


Figure 4.4 — Creating the Vehicle Customer custom object.

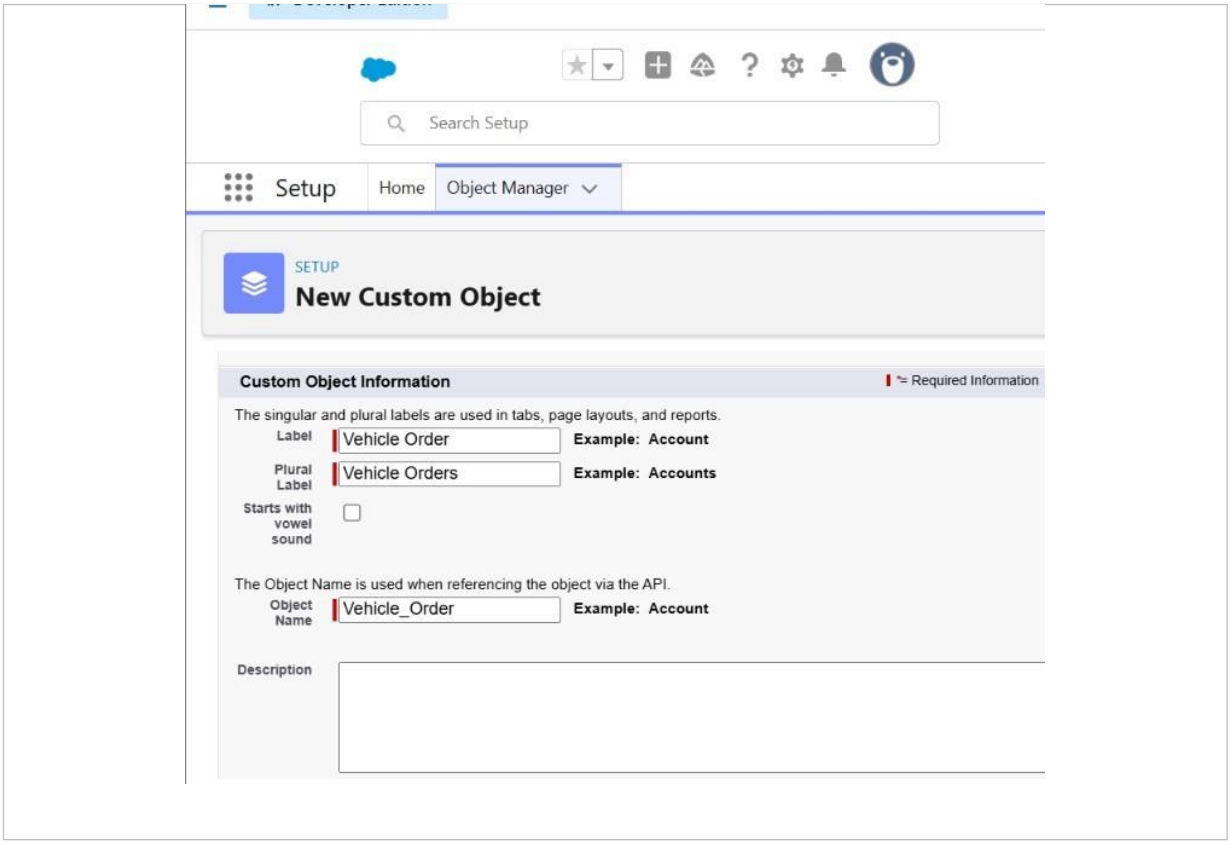


Figure 4.5 — Creating the Vehicle Order custom object.

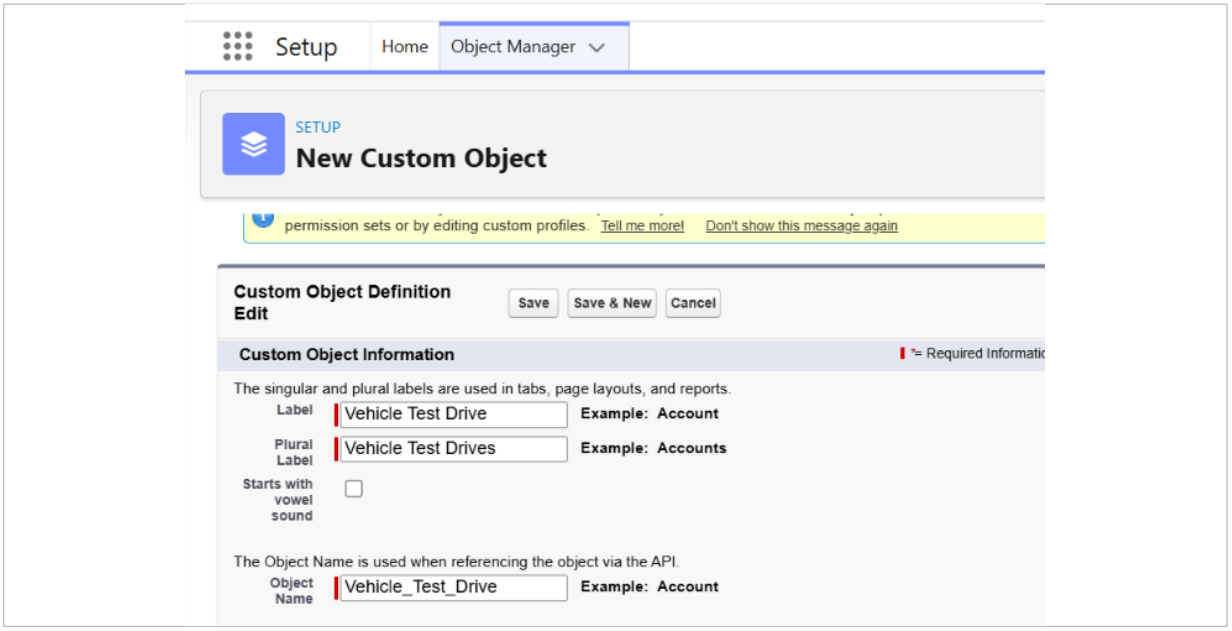


Figure 4.6 — Creating the Vehicle Test Drive custom object.

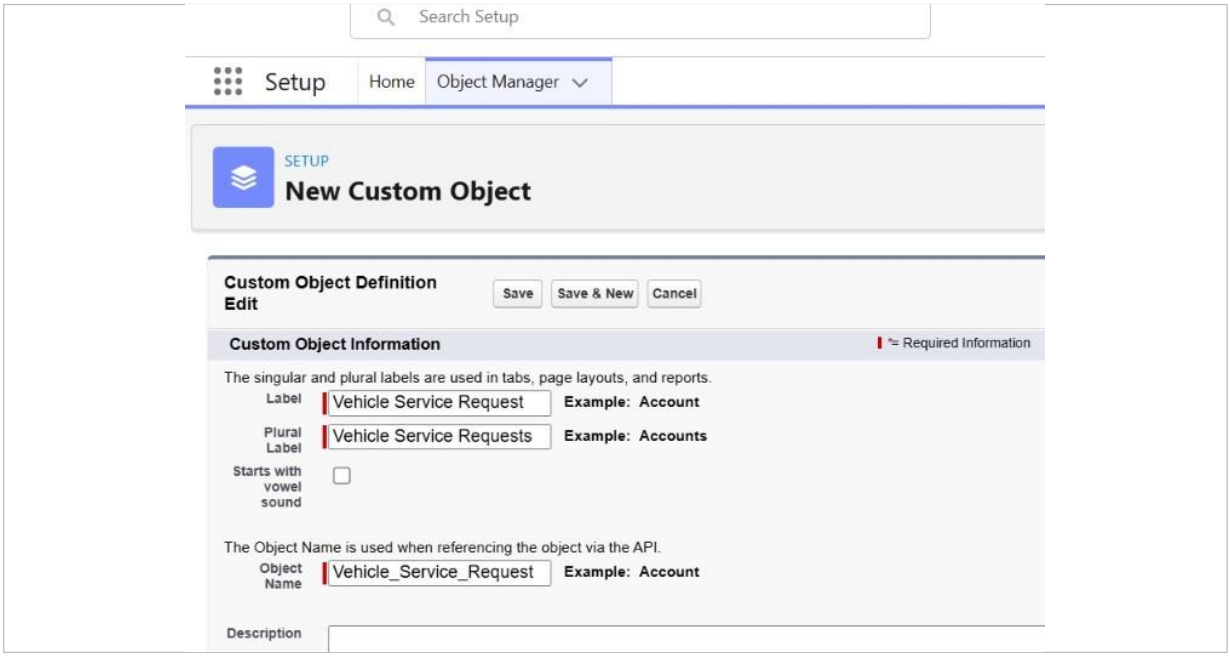


Figure 4.7 — Creating the Vehicle Service Request custom object.

4.2.1 Custom Tabs

A dedicated custom tab was created for every custom object so that each entity is independently navigable within the Lightning application.

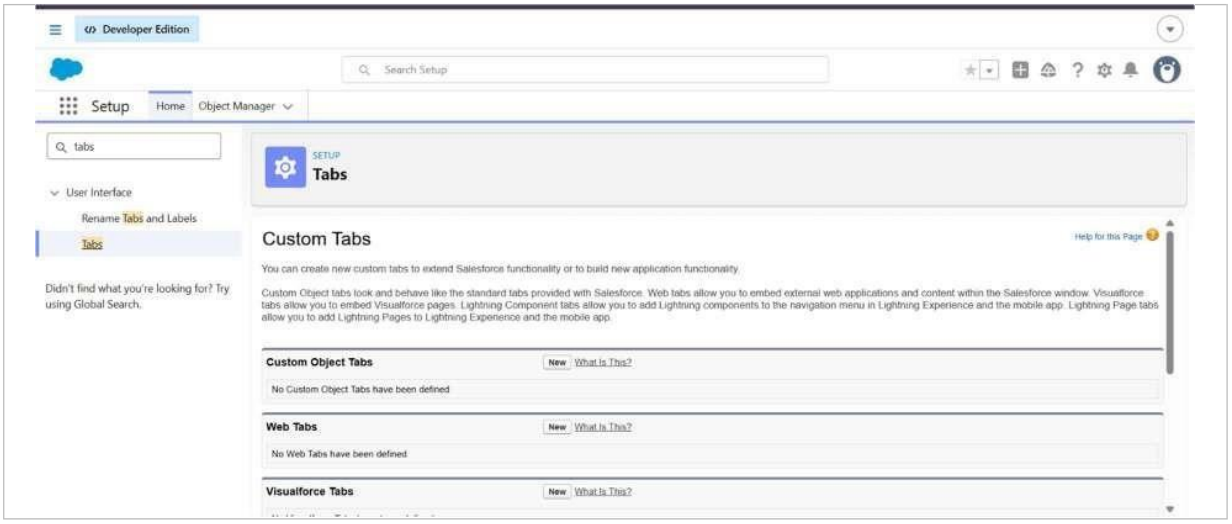


Figure 4.8 — The Custom Tabs setup page before tab creation.

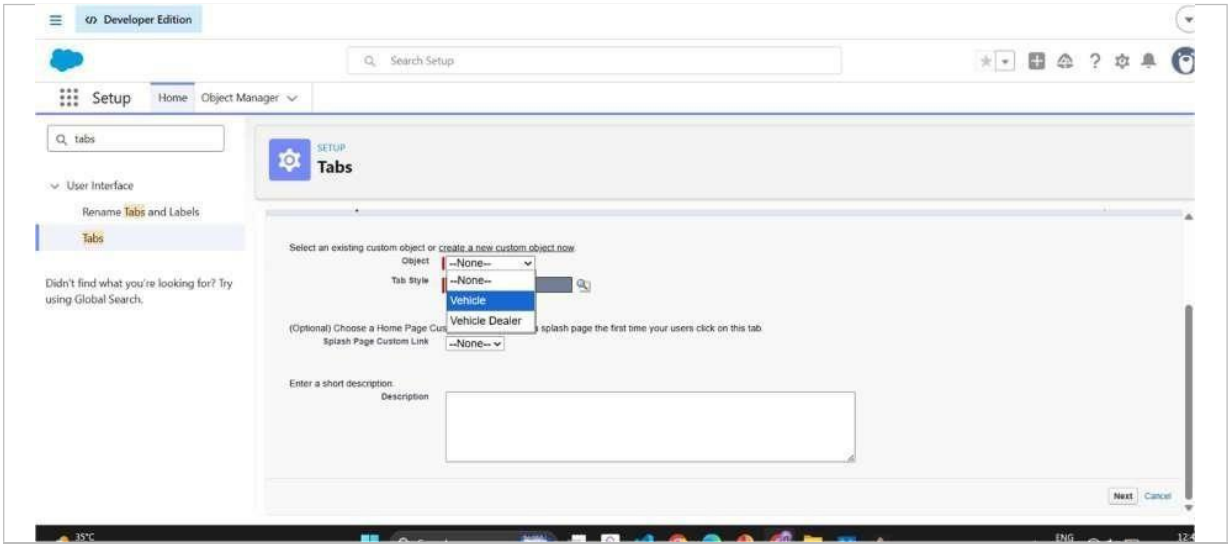


Figure 4.9 — Selecting the underlying object and tab style while creating a custom tab.

4.3 Fields

Custom fields were created on every object to capture exactly the information required by that entity, including text, number, currency, date, email, phone, picklist, and lookup fields. The final field sets are summarized below.

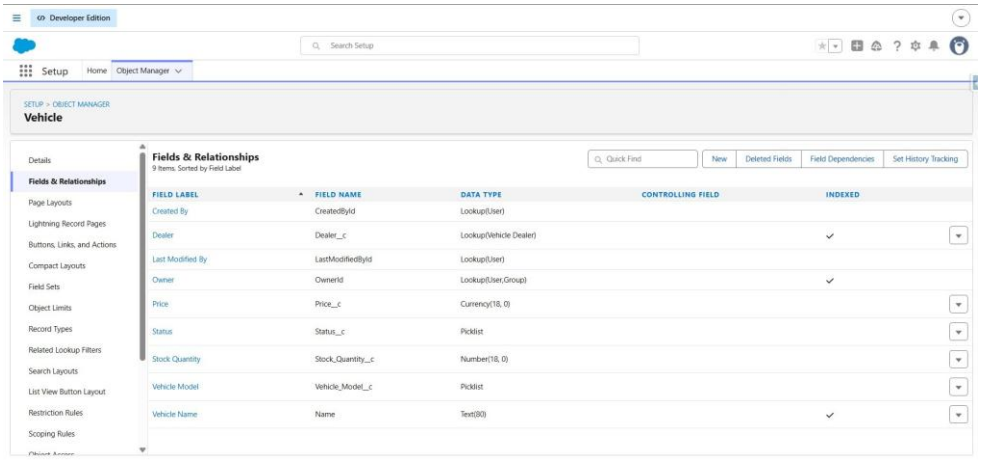


Figure 4.10 — Fields & Relationships for the Vehicle object.

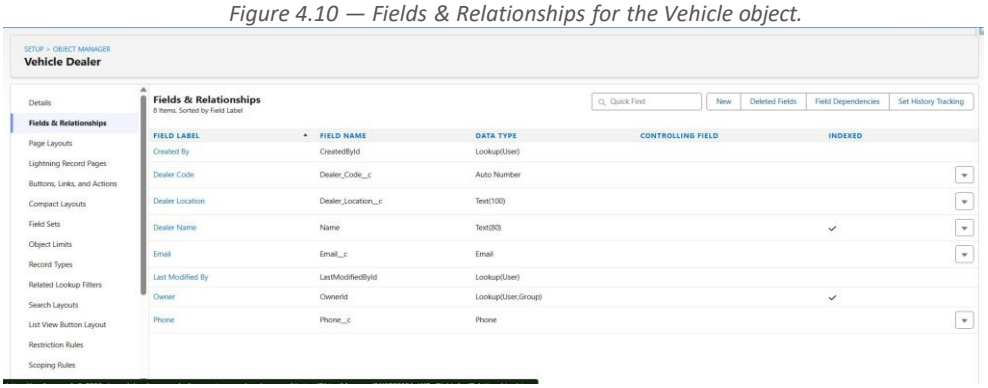


Figure 4.11 — Fields & Relationships for the Vehicle Dealer object.

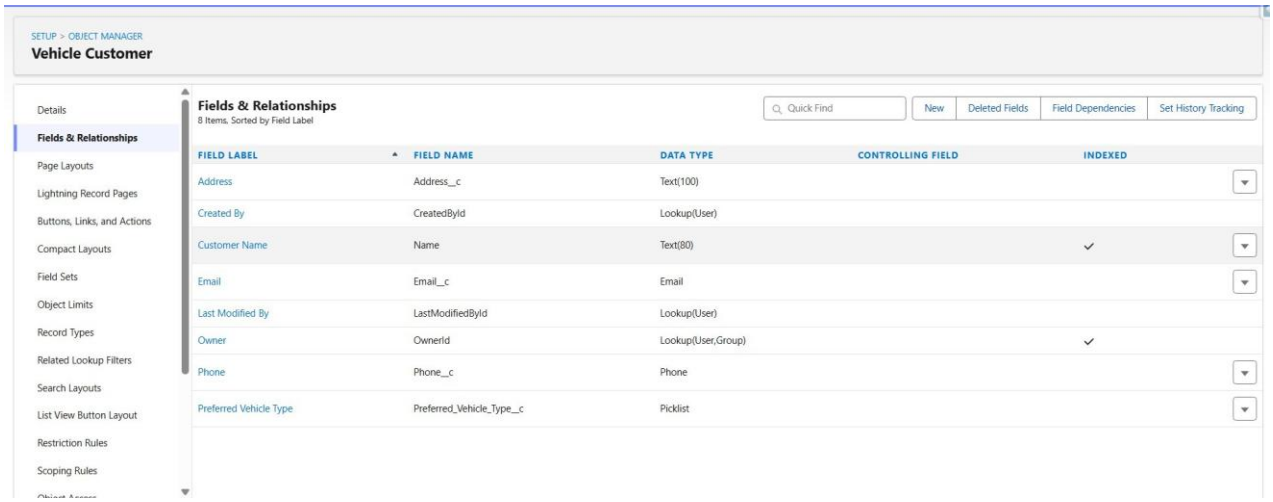


Figure 4.12 — Fields & Relationships for the Vehicle Customer object.

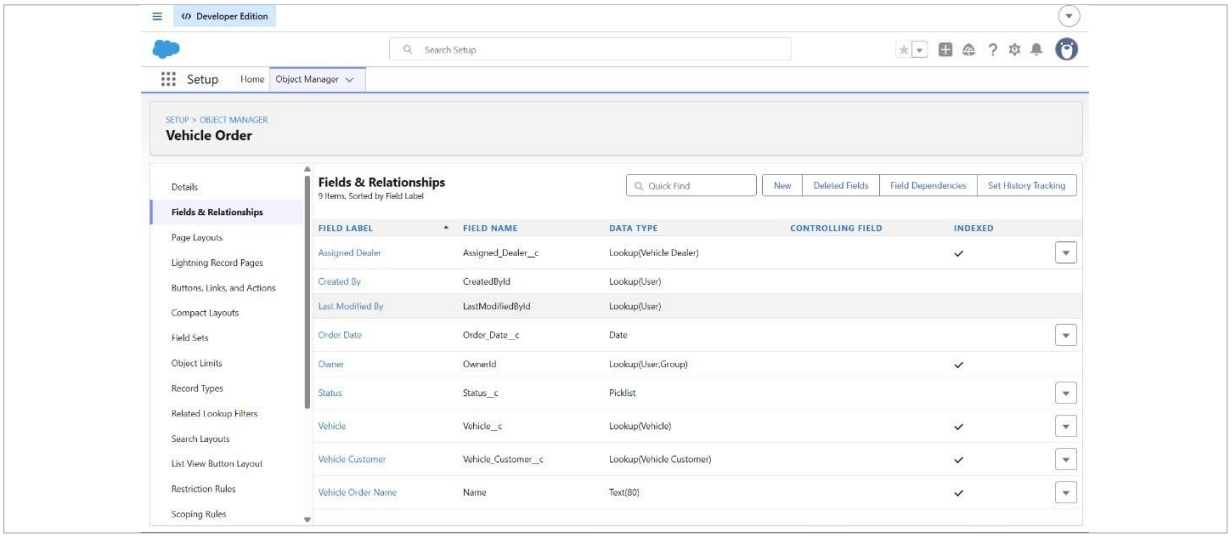


Figure 4.13 — Fields & Relationships for the Vehicle Order object.

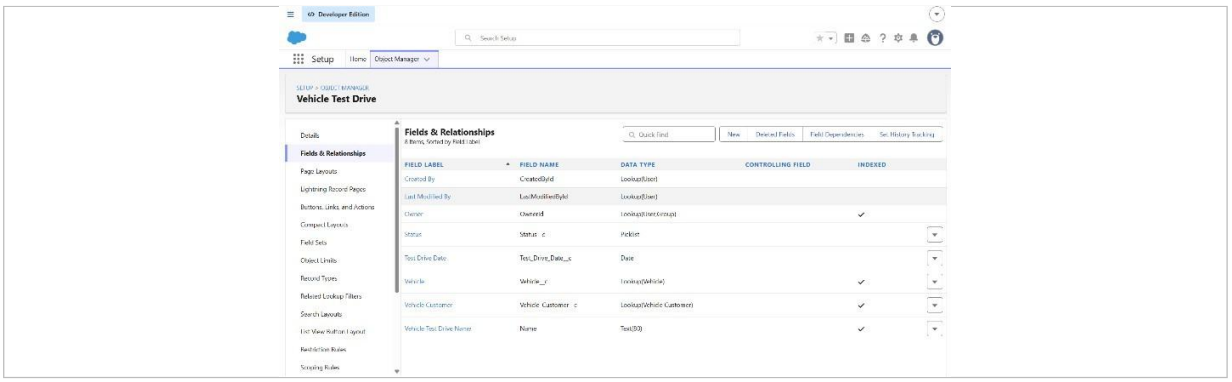


Figure 4.14 — Fields & Relationships for the Vehicle Test Drive object.

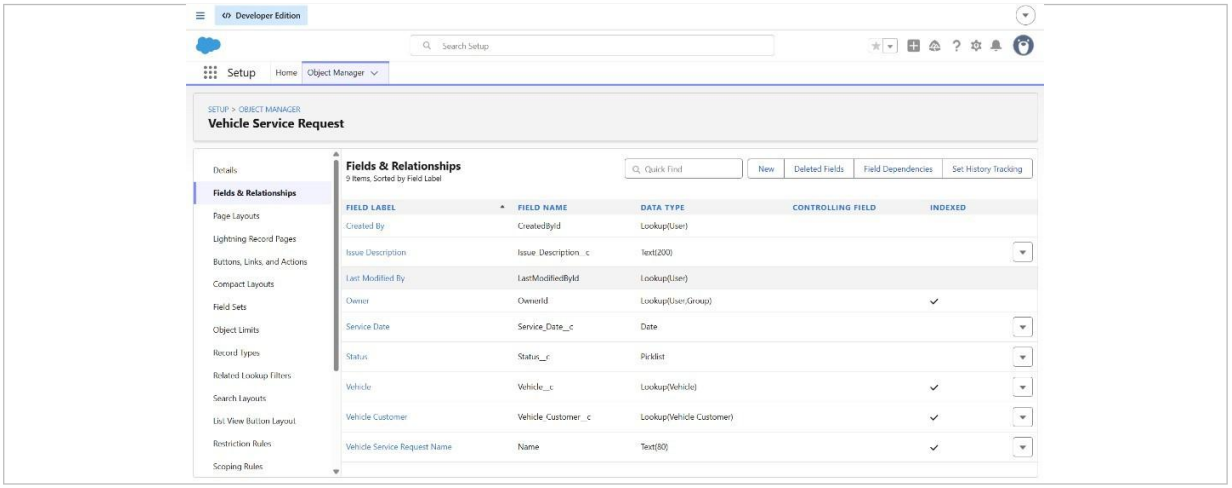


Figure 4.15 — Fields & Relationships for the Vehicle Service Request object.

4.3.1 Representative Field-Creation Walkthrough

The following sequence illustrates the standard field-creation process used across all objects, shown here for the Vehicle Model picklist field on the Vehicle object.

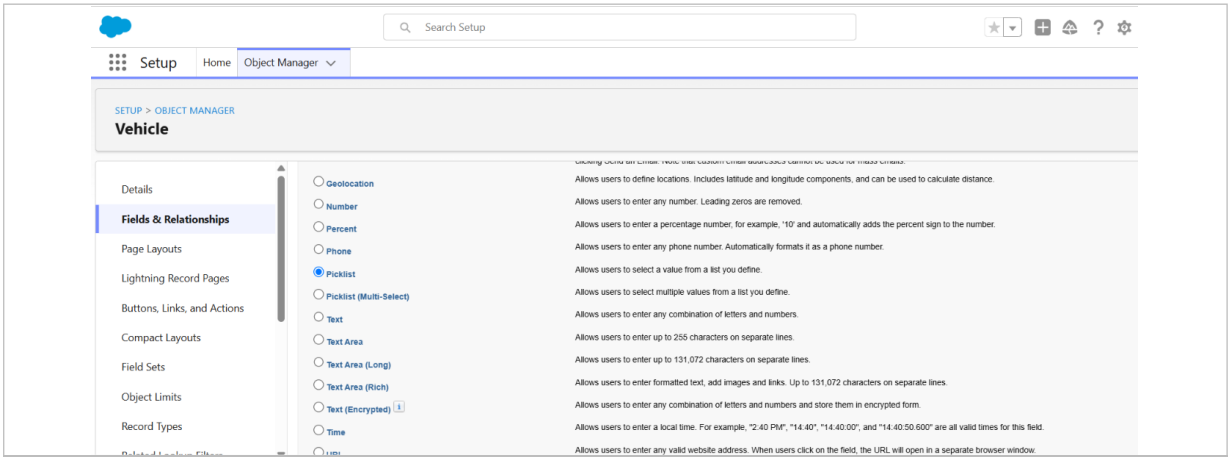


Figure 4.16 — Selecting the Picklist data type when creating a new custom field.

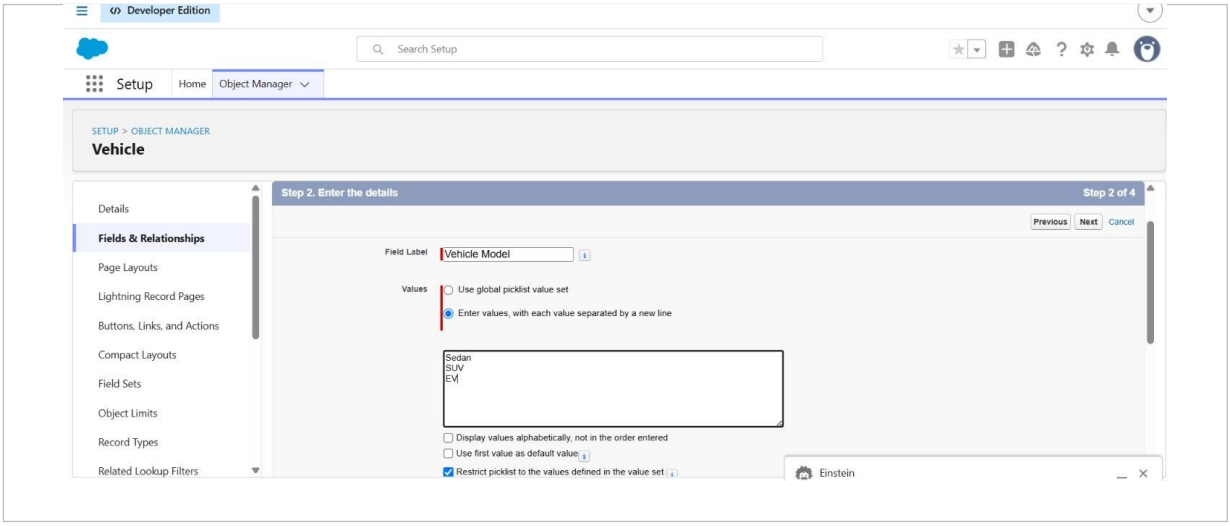


Figure 4.17 — Defining picklist values (Sedan, SUV, EV) for the Vehicle Model field.

4.4 Object Relationships

Lookup relationships connect Vehicle Orders and Test Drives to their related Vehicle and Vehicle Customer records, and connect Service Requests to the relevant Vehicle and Customer. The steps below illustrate the relationship-creation process.

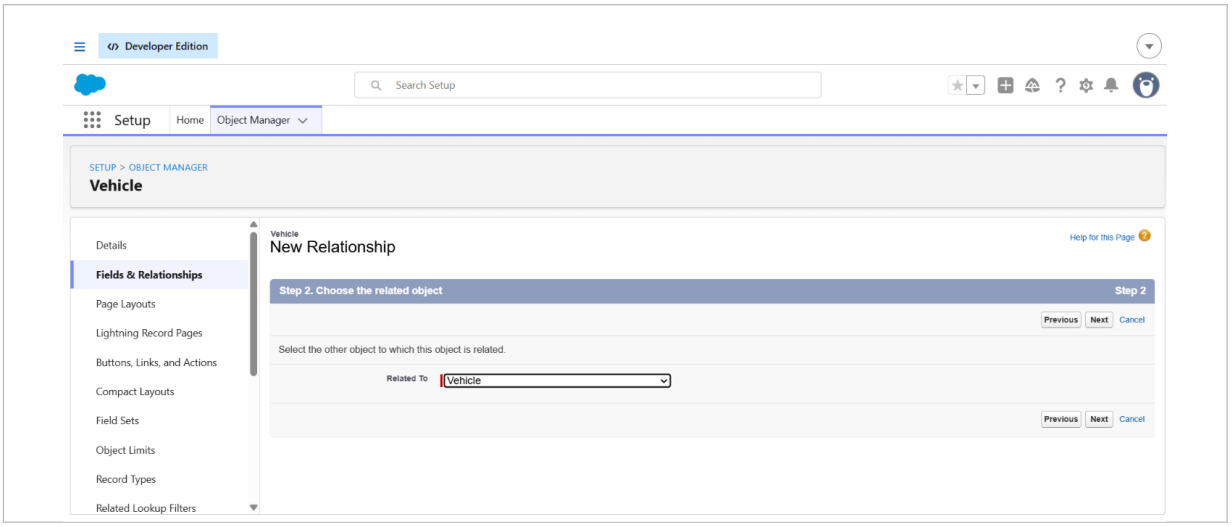


Figure 4.18 — Choosing the related object while creating a new lookup relationship.

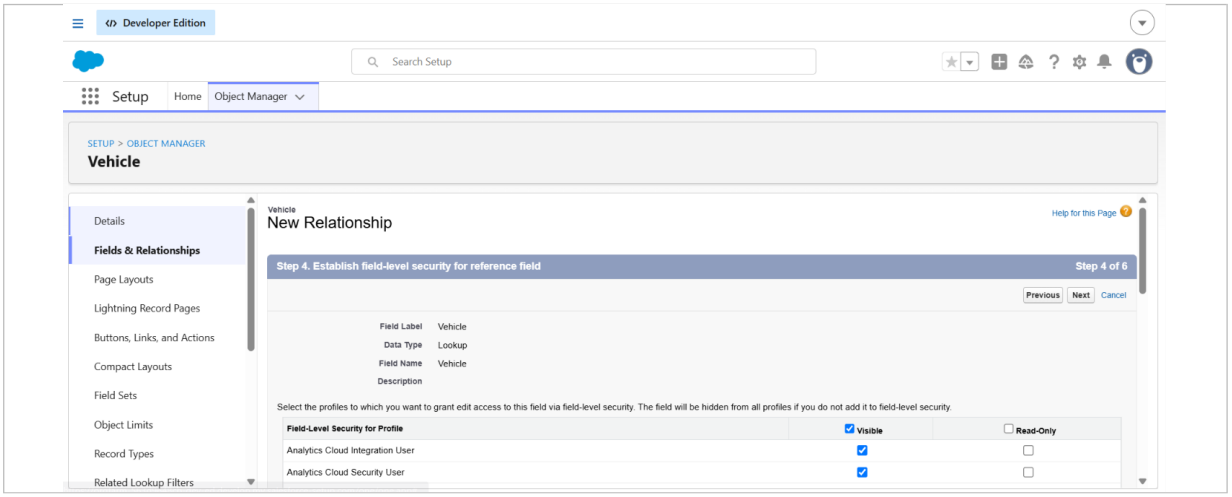


Figure 4.19 — Setting field-level security for the new relationship field.

4.5 Validation Rule — Vehicle Stock

A validation mechanism was implemented to ensure that an order cannot be confirmed when the associated vehicle does not have the required stock. This rule protects the ordering process from an invalid stock condition — for example, when the stock quantity of a selected vehicle is zero, any attempt to create a confirmed order is blocked and the configured validation message is displayed to the user.

4.6 Flow Automation

Salesforce Flow was used to automate selected business processes so that not every operation needs to be performed manually. Two record-triggered flows were built for this project.

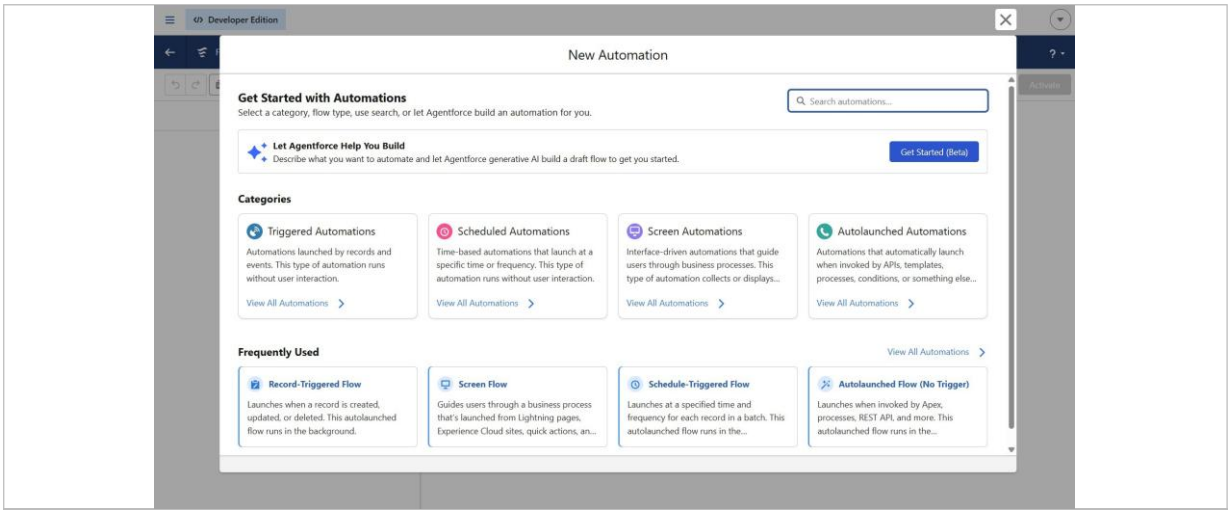


Figure 4.20 — The Flow Builder 'New Automation' dialog, showing the available automation categories.

4.6.1 Auto Assign Dealer

Triggered whenever a new Vehicle Order is created, this flow retrieves the customer's details, identifies the nearest available dealer, and updates the order with the assigned dealer — removing the need for manual dealer allocation.

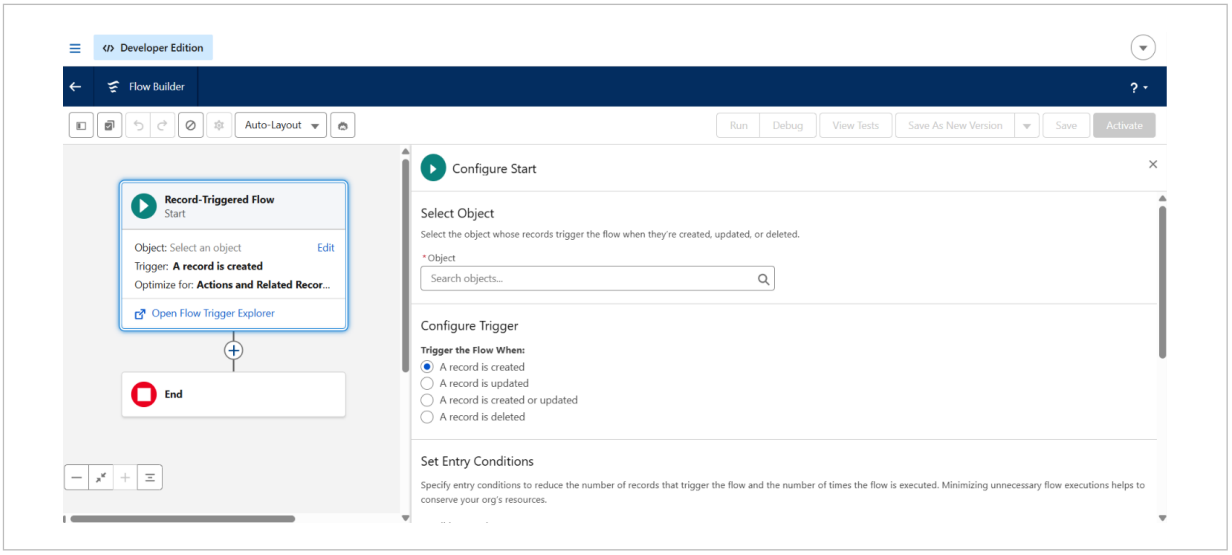


Figure 4.21 — Configuring the record-triggered start element for the Auto Assign Dealer flow.

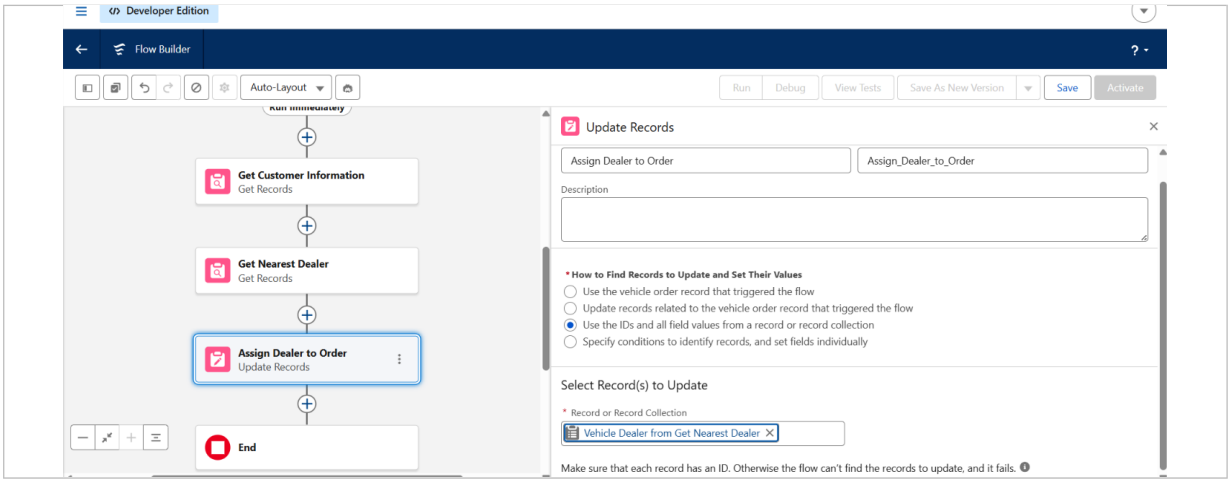


Figure 4.22 — The 'Assign Dealer to Order' update-records step.

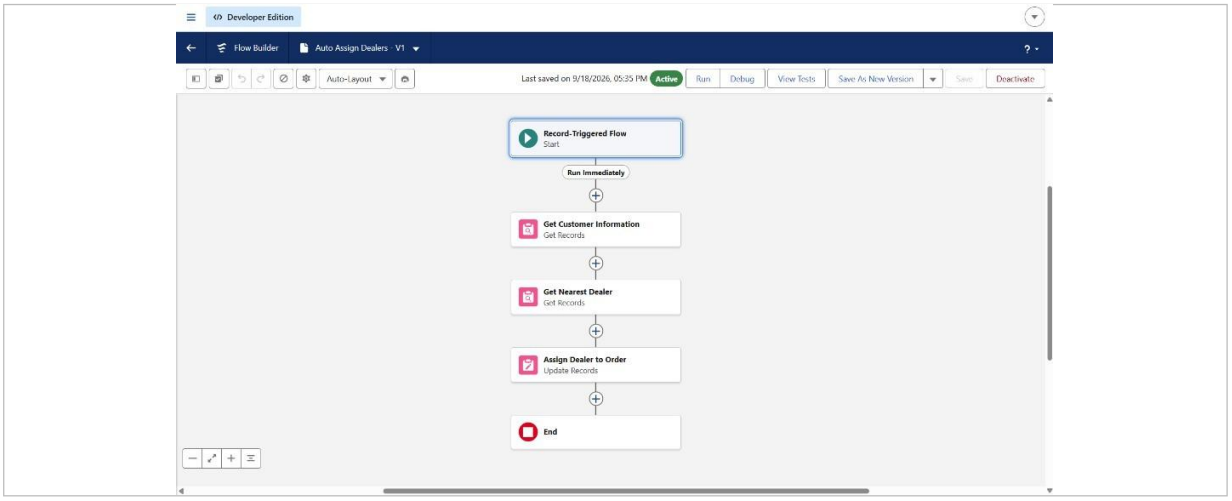


Figure 4.23 — The completed Auto Assign Dealer flow, active in the org.

4.6.2 Test Drive Reminder

This flow runs on the Vehicle Test Drive object and uses a scheduled path to send an automated reminder email to the customer one day before their booked test drive.

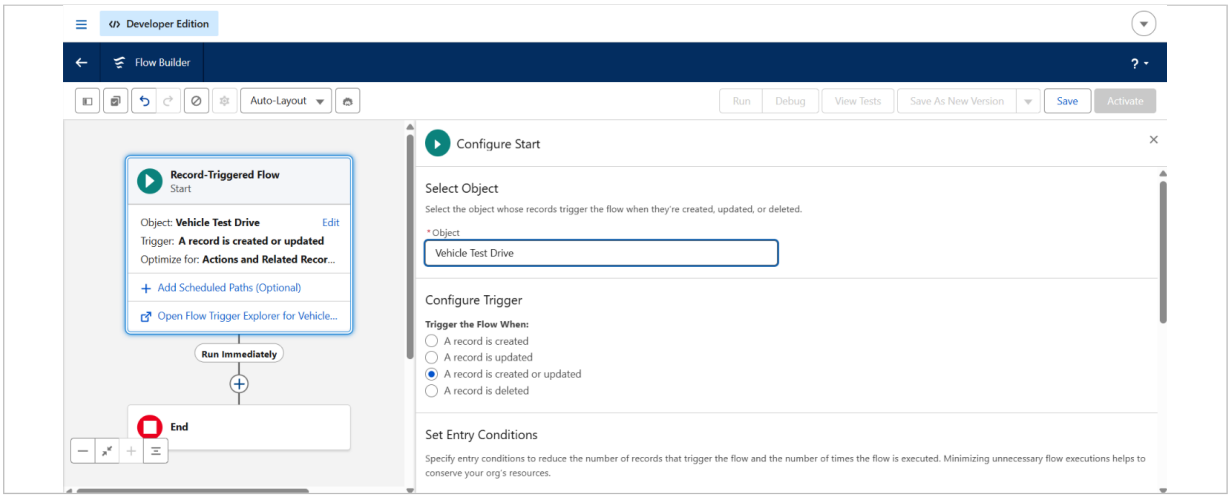


Figure 4.24 — Configuring the trigger for the Test Drive Reminder flow.

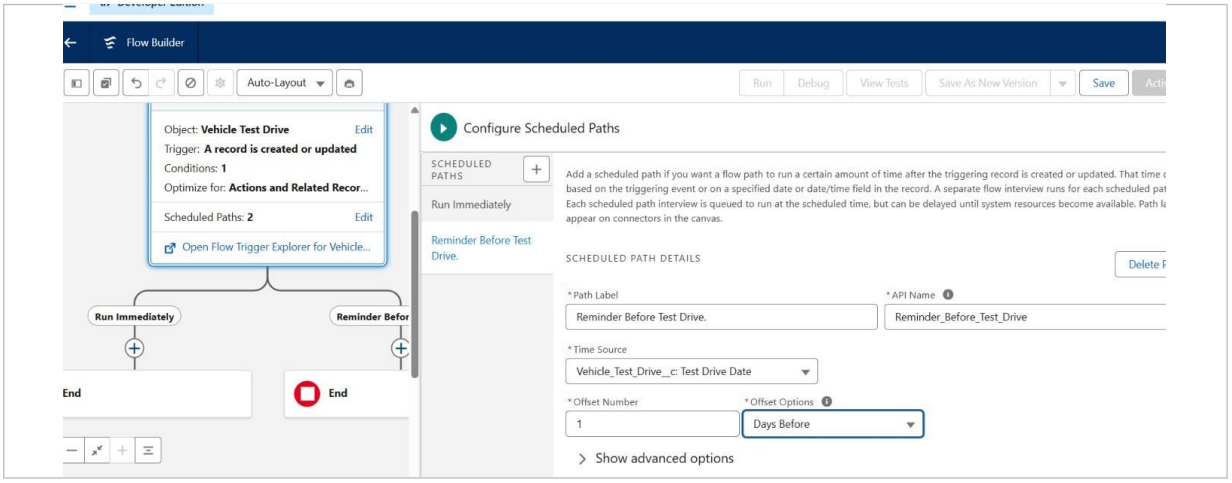


Figure 4.25 — The scheduled path, set to run one day before the Test Drive Date.

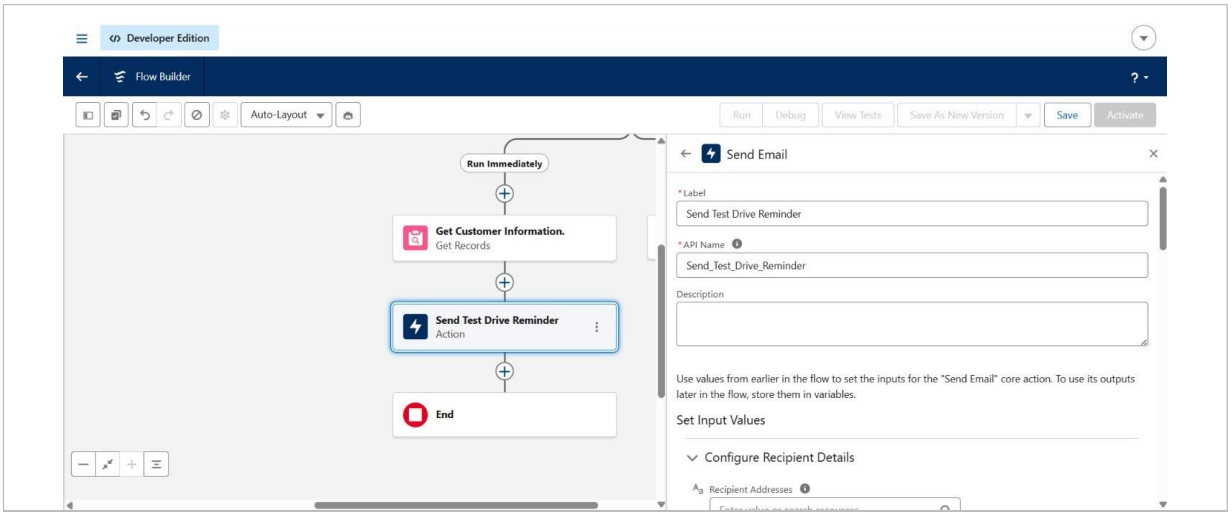


Figure 4.26 — The 'Send Test Drive Reminder' email action.

4.7 Apex Classes and Triggers

4.7.1 Apex Classes

- **Vehicle Order Processing:** Handles the business logic associated with vehicle orders.
- **Stock Validation:** Checks vehicle availability and ensures orders follow the defined stock conditions.
- **Order Management:** Supports processing and updating vehicle order records based on status.
- **Reusable Business Logic:** Logic is separated into Apex classes to keep the implementation easy to maintain and test.
- **Automation Support:** Apex classes work together with triggers and declarative automation to perform the required operations efficiently.

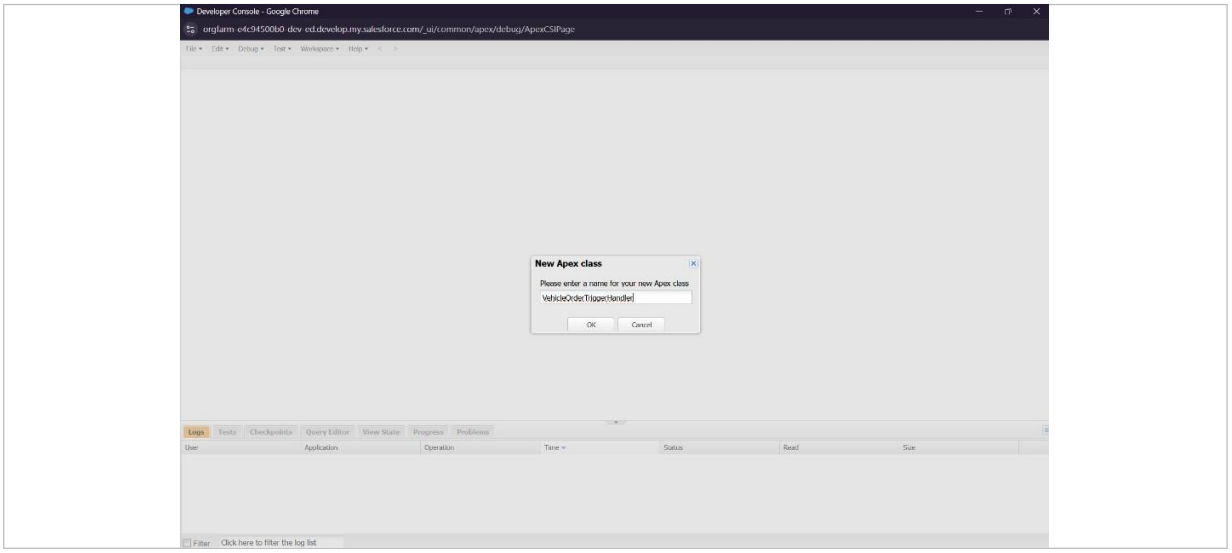


Figure 4.27 — Creating the VehicleOrderTriggerHandler Apex class in Developer Console.

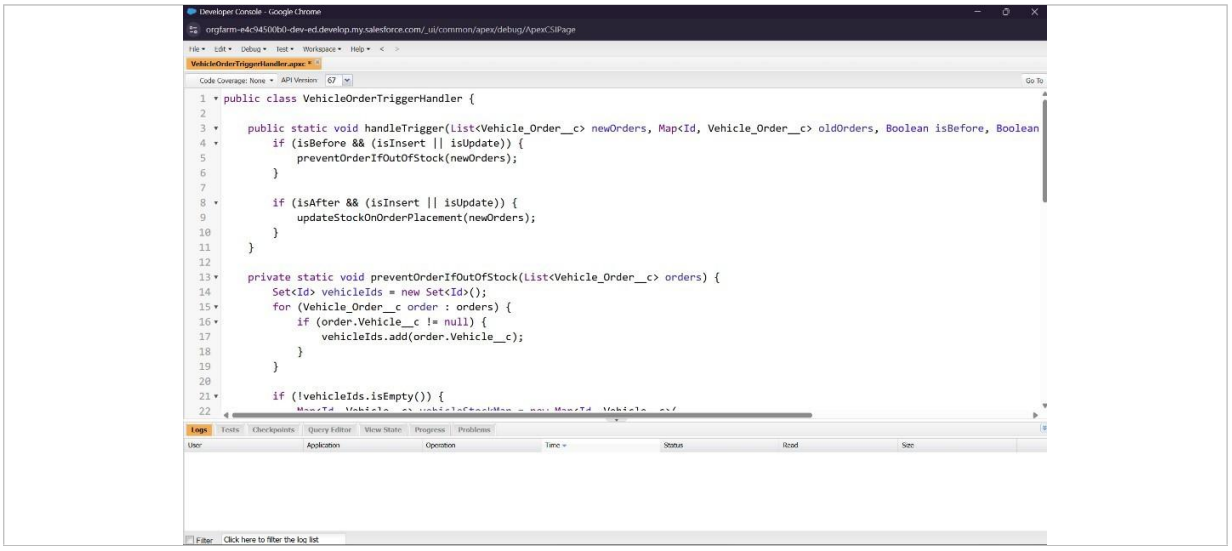


Figure 4.28 — The VehicleOrderTriggerHandler class, which centralizes the order-processing logic.

4.7.2 Apex Trigger

The vehicle order process is also backed by Apex trigger logic. The trigger responds to relevant order events and performs the configured business checks, including:

- Checking stock availability.
- Preventing invalid order processing.
- Updating vehicle stock after a confirmed transaction.
- Supporting order-status processing.

The trigger logic is organized using a handler pattern, so that the business logic remains separated from the trigger definition itself — a best practice that improves testability and maintainability.

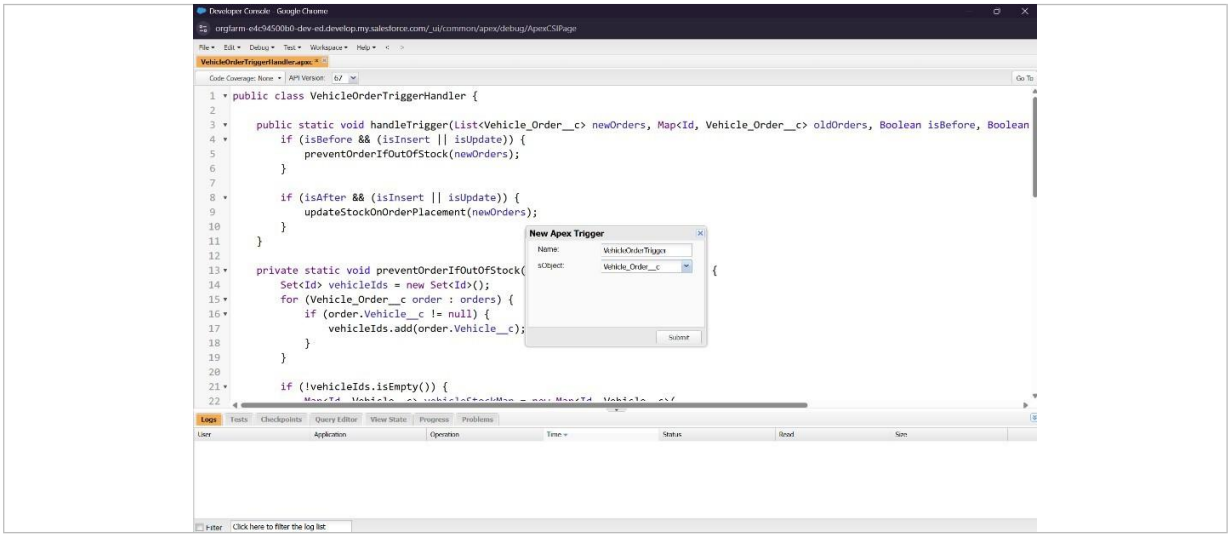


Figure 4.29 — Creating the VehicleOrderTrigger, bound to the Vehicle_Order__c object.

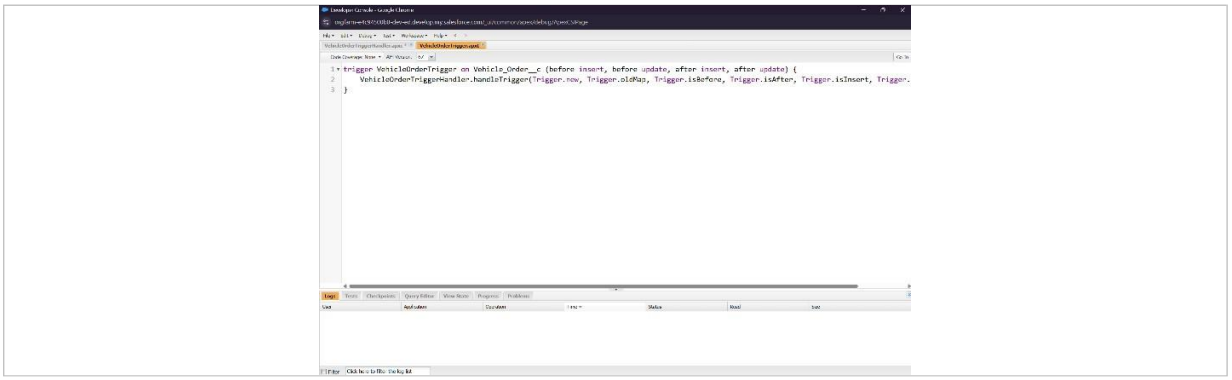


Figure 4.30 — The VehicleOrderTrigger definition, delegating to the handler class.

5. Phase 3 — UI/UX Development & Customization

The third phase focused on building a practical, cohesive user interface within Salesforce Lightning, covering the Lightning App, App Manager, page layouts, dynamic forms, user management, reports, dashboards, and Lightning record pages.

5.1 Lightning App Setup via App Manager

The WhatNext Vision Motors Lightning App provides a centralized interface for managing Vehicles, Dealers, Customers, Orders, Test Drives, and Service Requests, improving navigation and giving users quick access to the records and functions they need.

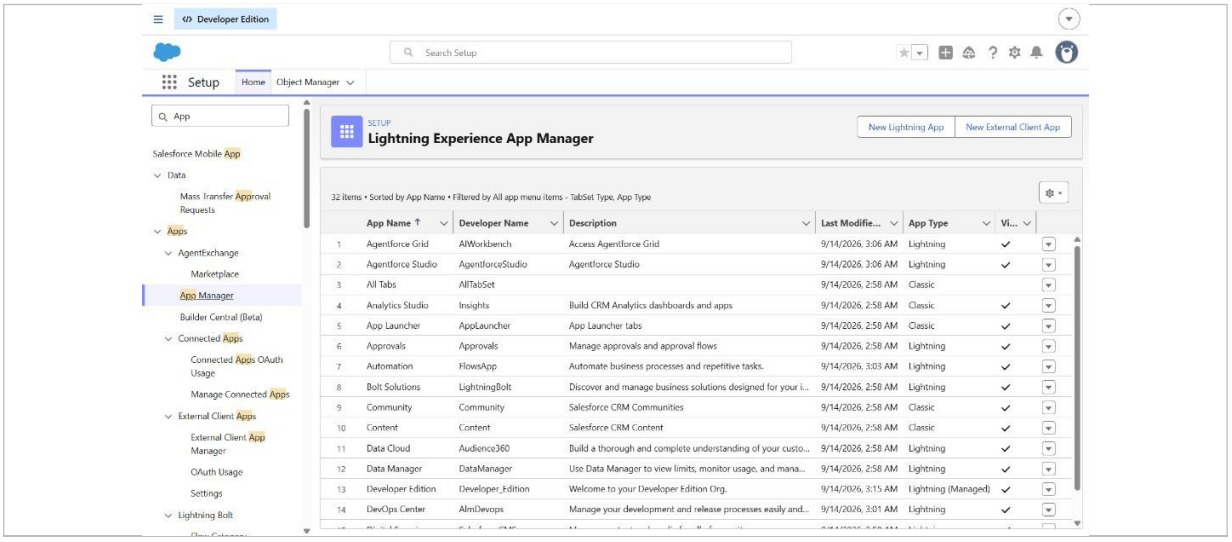


Figure 5.1 — The Lightning Experience App Manager, before the new app is created.

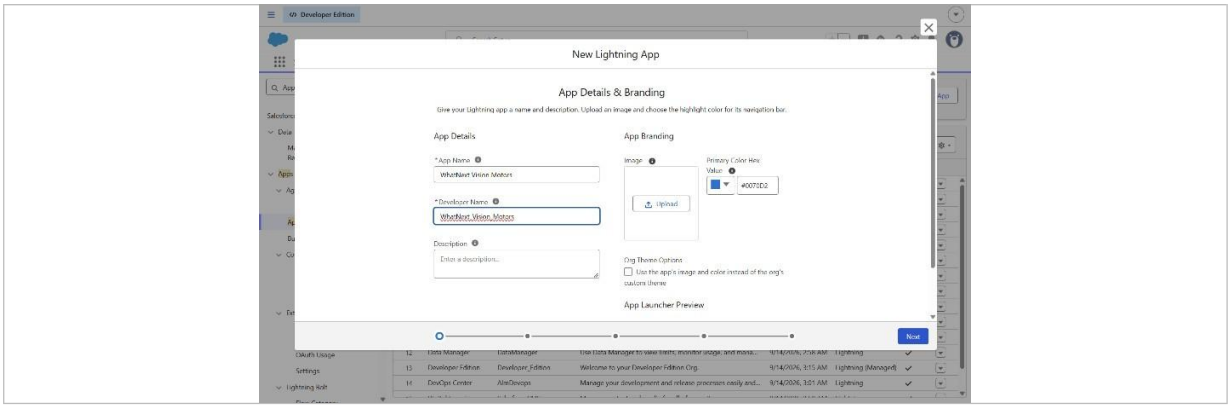


Figure 5.2 — Naming and branding the new 'WhatNext Vision Motors' Lightning App.

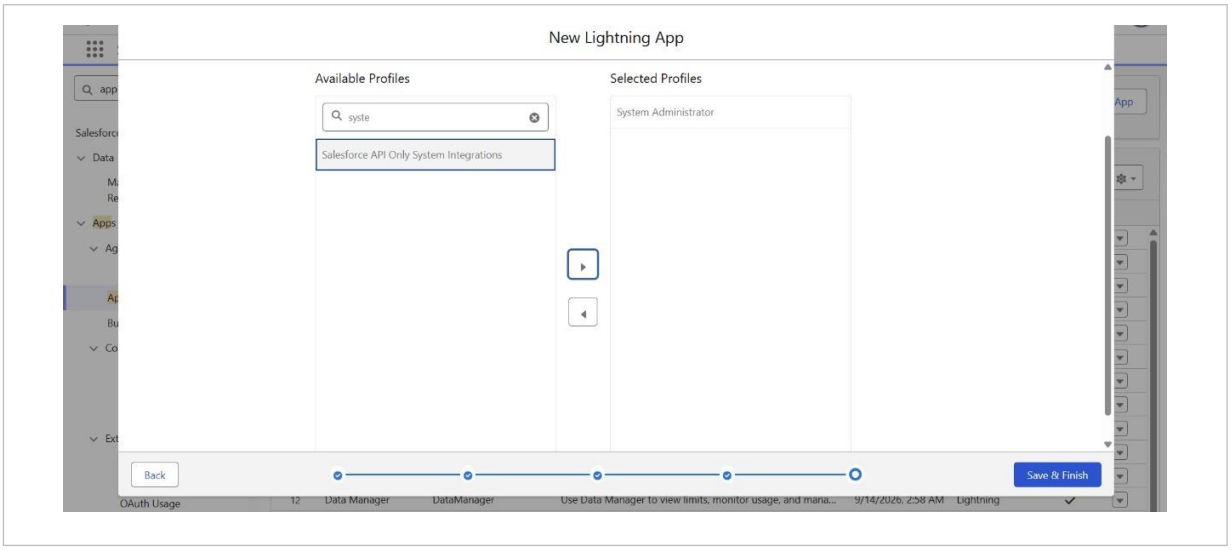


Figure 5.3 — Assigning user profiles that can access the new app.

5.2 Page Layouts & Dynamic Forms

Page layouts were configured to arrange fields and related information on each record, giving users access to relevant information while keeping the record interface organized. Dynamic Forms were used where conditional field visibility was required, so that fields are displayed according to the values or conditions associated with a record rather than all at once.

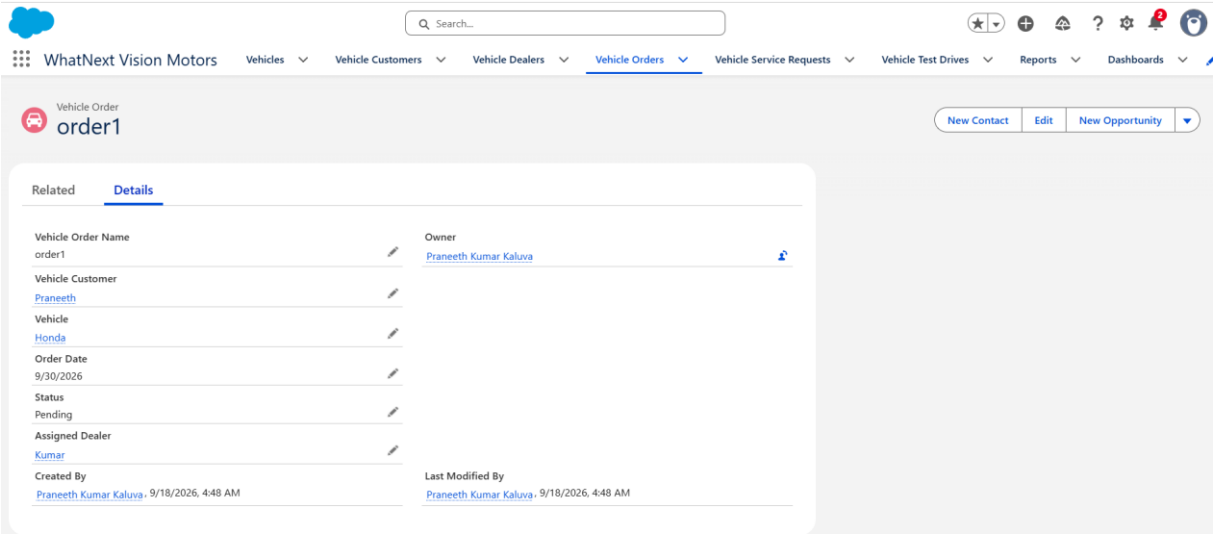


Figure 5.4 — A Vehicle Order record page, laid out for quick review of order details.

5.3 User Management

Users and records were configured according to the access requirements of the application, and record-entry forms were verified for usability by capturing sample data.

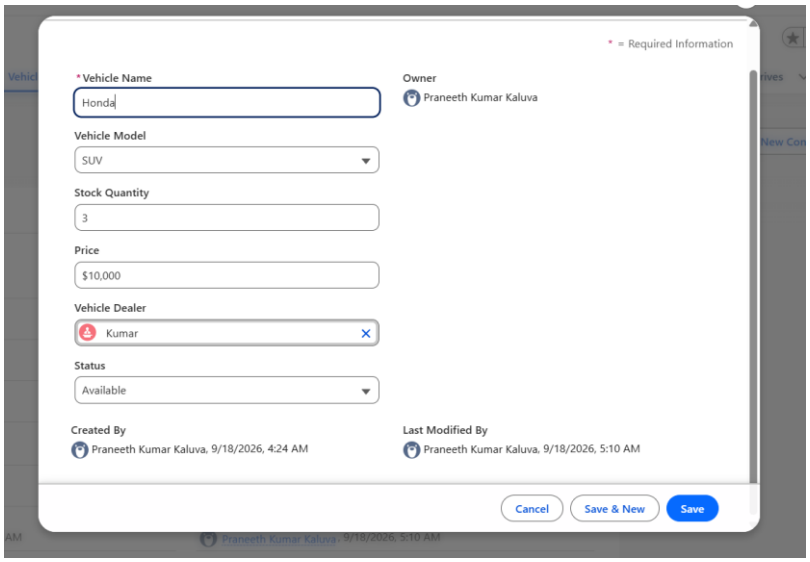


Figure 5.5 — Creating a new Vehicle Customer record through the Lightning form.

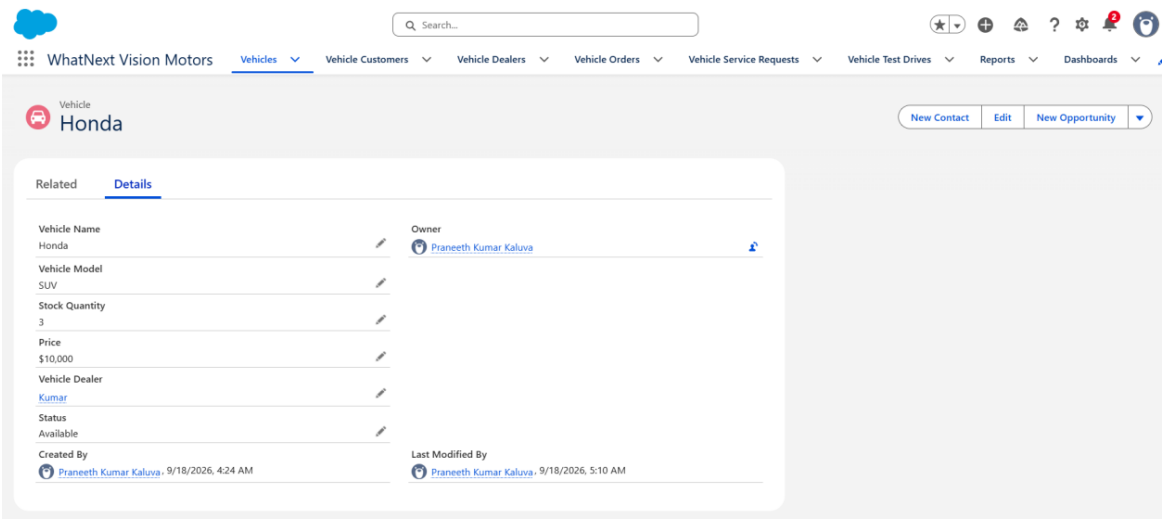


Figure 5.6 — The saved Vehicle Customer record.

5.4 Reports and Dashboards

Reports were configured to provide structured views of Salesforce records — for example, vehicle stock levels, order status distribution, and test-drive completion rates. Dashboards provide a visual representation of this report information, helping users quickly understand business activity without manually reviewing individual records.

5.5 Lightning Record Pages

Lightning Record Pages were configured to provide a useful, at-a-glance view of individual records, combining record details, the highlights panel, and related lists on a single page.

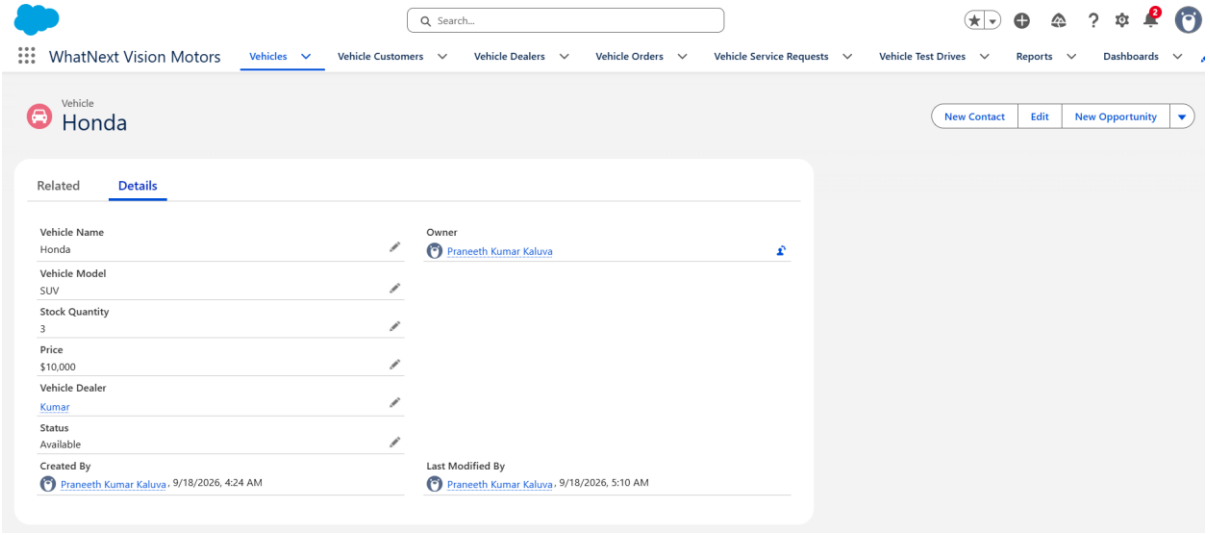


Figure 5.7 — A Vehicle record page showing related Vehicle Orders and Vehicle Test Drives.

6. Phase 4 — Data Migration, Testing & Security

The fourth phase focused on populating the CRM with data, verifying implemented functionality, and configuring security controls: data loading, profiles, roles, permission sets, sharing rules, and structured testing.

6.1 Data Loading

Initial data was prepared in a structured format before being added to Salesforce. The data-loading process involved:

- Preparing sample records.
- Mapping data fields to Salesforce fields.
- Loading the required records.
- Checking whether the records were created correctly.
- Verifying relationships between associated records.

Where applicable, the Salesforce Data Import Wizard or Data Loader can be used depending on the object and volume of records.

6.2 Profiles

Profiles provide the basic permission structure for Salesforce users, determining the level of access users have to applications, objects, fields, and other platform functionality

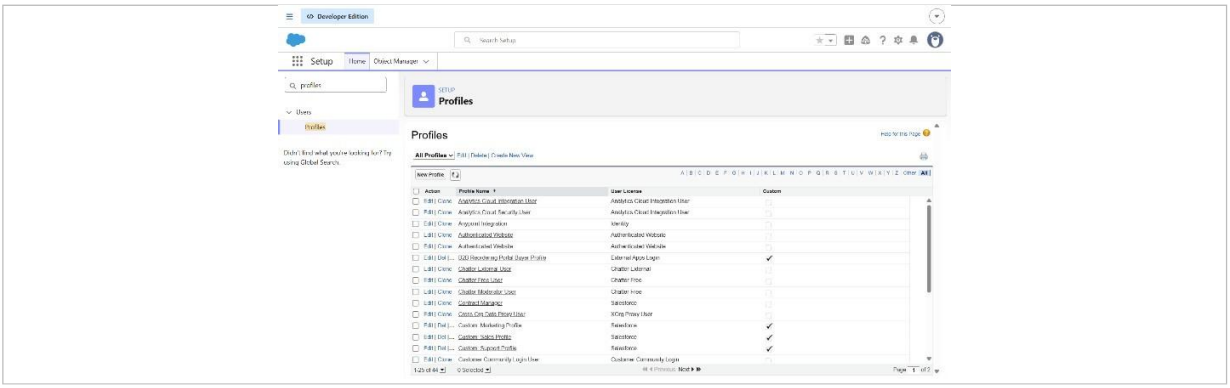


Figure 6.1 — The Profiles list in Setup.

6.3 Roles and Role Hierarchy

The role hierarchy represents the organizational structure used for record-access considerations. A representative structure for this project is:

CEO → Sales Manager → Sales Representative

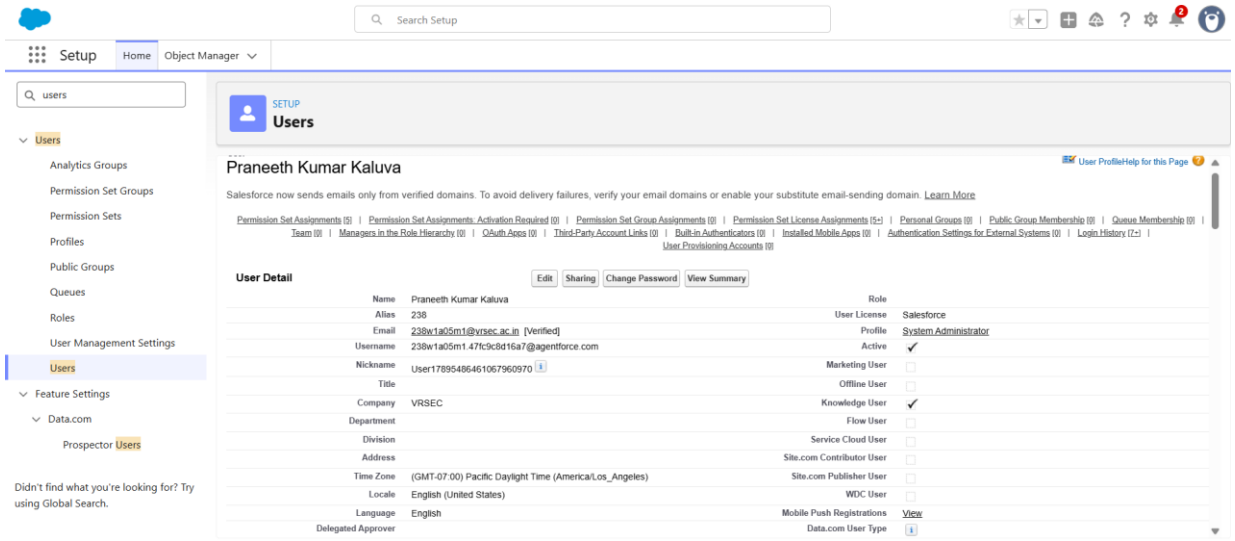


Figure 6.2 — Assigning a role to a user record in the org.

6.4 Permission Sets

Permission Sets were used to grant additional access to users who require specific Salesforce functionality beyond what their profile provides.

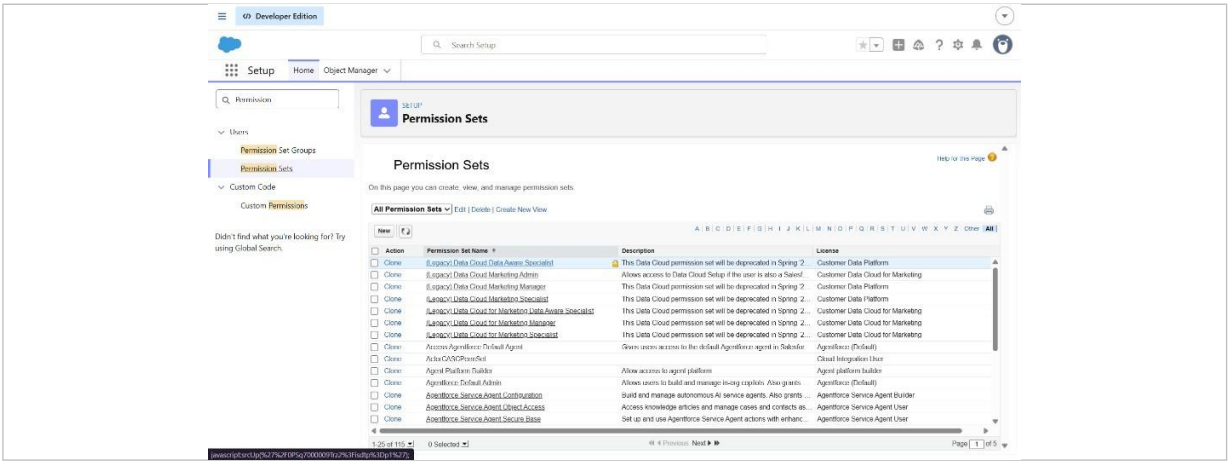


Figure 6.3 — The Permission Sets list in Setup.

6.5 Sharing Rules

Sharing configuration determines how records are accessed by different users. The sharing model is aligned with the organization's responsibilities so that users can work with the information required for their role while avoiding unnecessary exposure of restricted records.

6.6 Test Cases

Test cases were prepared for the core Salesforce features — record creation, validation, automated flows, and triggers — each supported by input and output evidence.

Test Case 1 — Vehicle Creation

Objective: Verify that a valid vehicle record can be created.

Test Data:

- Vehicle Name: Honda
- Model: SUV
- Stock Quantity: 98
- Price: $1,200,000
- Status: Available
- Dealer: Maruthi

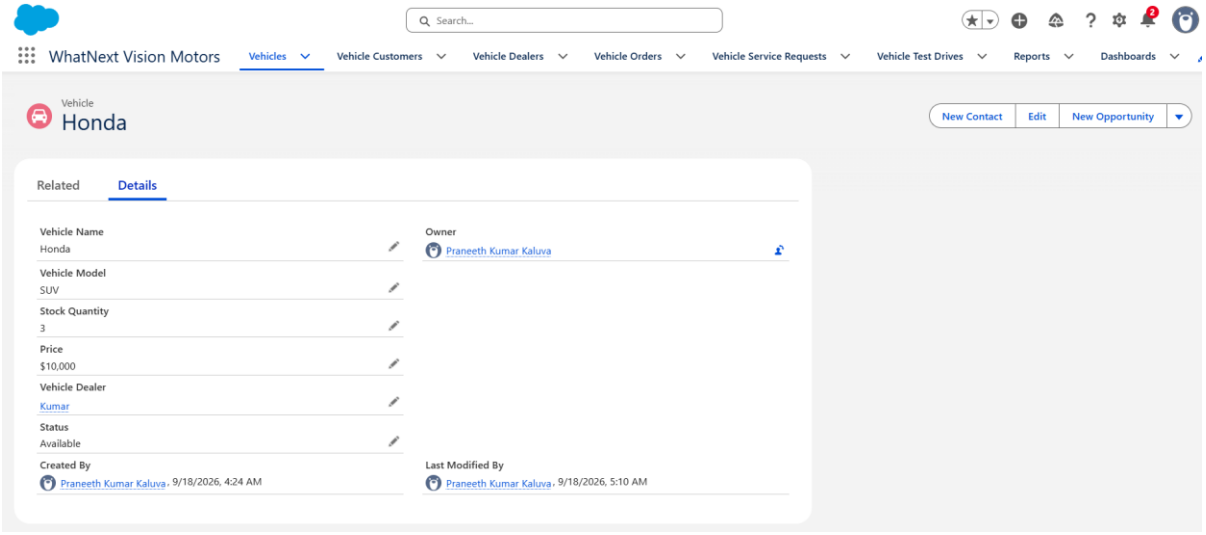


Figure 6.4 — The created Vehicle record, confirming all fields saved correctly.

Test Case 2 — Stock Validation on Order Confirmation

Objective: Verify that the stock validation rule correctly governs order confirmation, allowing a Confirmed status only when the associated vehicle has available stock.

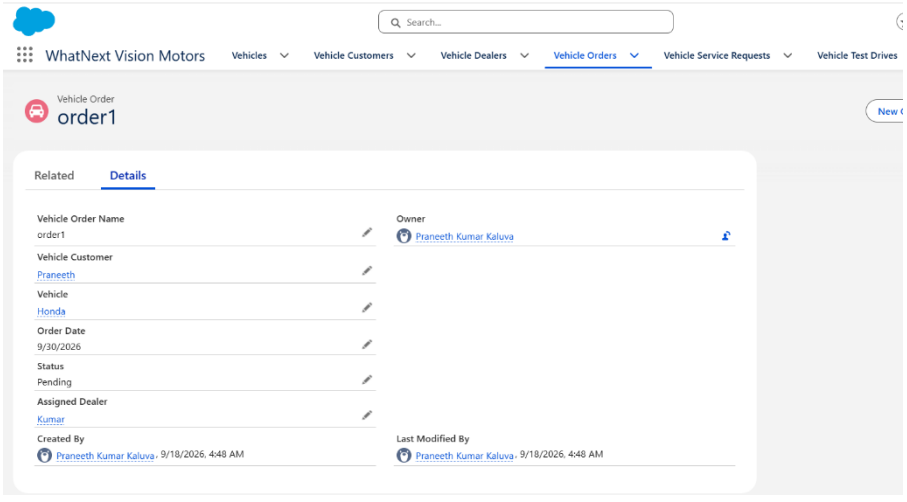


Figure 6.5 — A Vehicle Order confirmed once the associated vehicle's stock was validated as available

Test Case 3 — Test Drive Reminder Email

Objective: Verify the automated reminder process for an upcoming test drive.

Test Data:

- Customer: Existing customer
- Customer Email: Valid email
- Status: Scheduled
- Test Drive Date: Following day

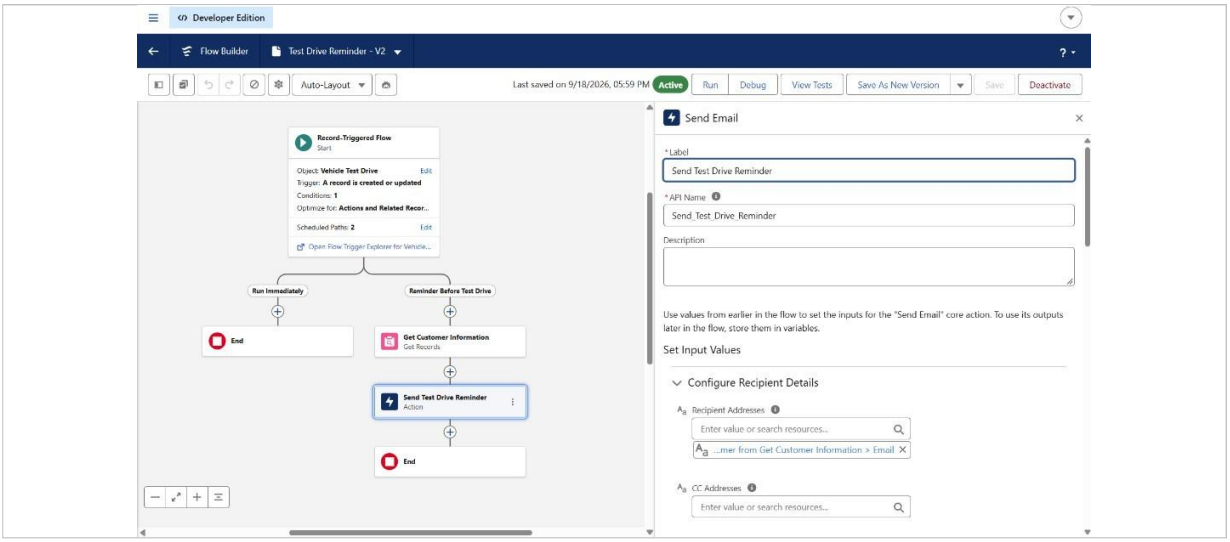


Figure 6.6 — Configuring the 'Send Email' action used by the reminder flow

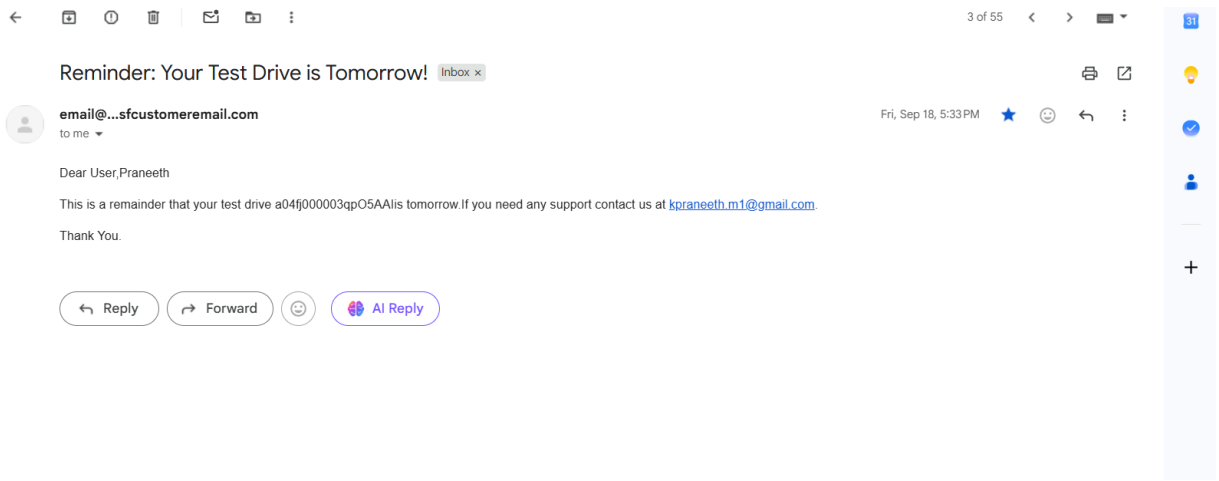


Figure 6.7 — The resulting reminder email received by the customer, confirming the automation worked end-to-end

Test Case 4 — Test Drive Completion

Objective: Verify that a test drive record can progress through its lifecycle and be marked Completed.

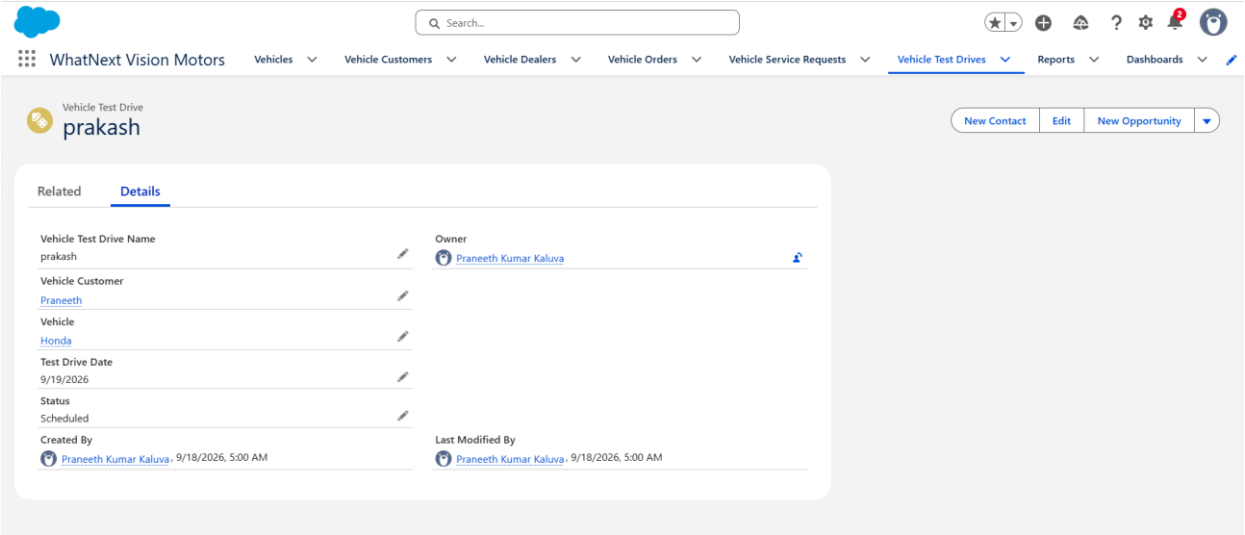


Figure 6.8 — A Vehicle Test Drive record with status updated to Completed.

7. Phase 5 — Deployment, Documentation & Maintenance

The final phase covers moving Salesforce components to their intended environment, monitoring the application, handling issues, and maintaining the system over time.

7.1 Deployment Strategy

Salesforce metadata can be transferred between environments using supported deployment mechanisms (e.g., Change Sets, packages, or CI/CD tooling). For this project, the deployment package includes:

- Custom objects
- Custom fields
- Relationships
- Validation rules
- Flows
- Apex classes
- Apex triggers
- Lightning pages
- Reports and dashboards

7.2 Maintenance & Monitoring

Salesforce's built-in monitoring tools help administrators identify problems in automated processes. Important areas to monitor include:

- **Apex Jobs:** Used to monitor batch and scheduled Apex execution.
- **Debug Logs:** Useful for investigating unexpected behaviour in Apex and other Salesforce operations.
- **Flow Errors:** Flow-failure information helps identify problems in automated processes.
- **User Access:** Periodic permission reviews help ensure users retain only the access they need.

7.3 Troubleshooting Approach

A structured troubleshooting method is followed whenever an issue is identified:

- Step 1 — Identify the Problem: Determine which Salesforce feature is producing the unexpected result.
- Step 2 — Reproduce the Issue: Repeat the same operation using controlled test data.
- Step 3 — Review Configuration: Inspect the relevant object, fields, validation rules, Flow, permissions, or Apex logic.
- Step 4 — Check Logs: Review available debugging information for programmatic or automated processing.

- Step 5 — Correct the Issue: Modify the configuration or code responsible for the problem.
- Step 6 — Retest: Run the original scenario again and verify that the expected result is obtained.
- Step 7 — Deploy the Correction: After successful testing, move the corrected configuration to the required environment using the chosen deployment process.

8. Conclusion

The WhatNext Vision Motors project resulted in a Salesforce CRM environment capable of organizing several automotive business activities within a single, connected platform. The implementation demonstrates how Salesforce can be customized to represent business-specific entities and processes end to end.

The major outcomes of the project are:

- Automotive records are organized using purpose-built Salesforce objects.
- Relationships connect customers, vehicles, dealers, orders, and related activities.
- Vehicle stock conditions are checked automatically during order processing.
- Dealer assignment is automated through Flow.
- Test-drive reminders are generated automatically ahead of scheduled appointments.
- Lightning pages provide a structured, intuitive user experience.
- Layered security controls provide appropriately scoped access.
- Reports and dashboards support ongoing business monitoring.
- Documented testing provides evidence that the implemented functionality works according to the defined scenarios.

Beyond the functional outcomes, the project provided practical, hands-on experience in combining Salesforce's declarative tools (Objects, Flow, Validation Rules) with programmatic development (Apex classes and triggers) — a combination that reflects real-world Salesforce Platform Developer work.